

MPI

A collage of several tutorials

Mirko Cestari

m.cestari@cineca.it

High Performance Computing department

Parallel programming with MPI

Introduction and Point-to-Point Communications



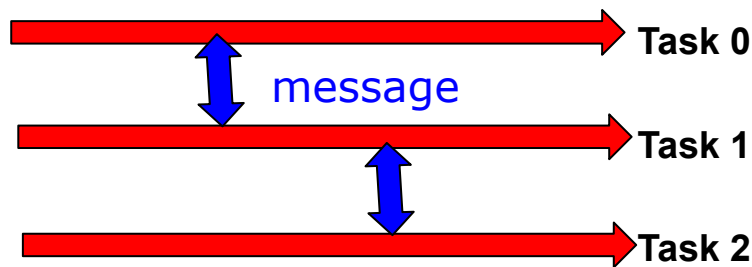
Distributed Memory Programming with MPI

Moreno Marzolla
Dip. di Informatica—Scienza e Ingegneria (DISI)
Università di Bologna

moreno.marzolla@unibo.it

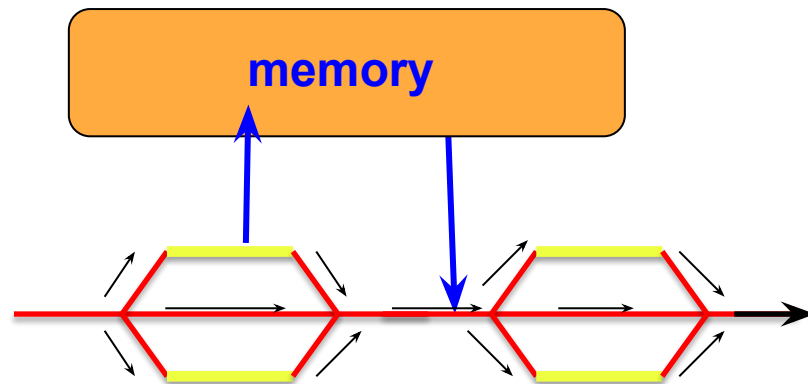
Message passing and shared memory parallelism

message passing



Multiple tasks exchange data via explicit messages

shared memory



Program splits into threads which share data via variables in shared memory

Message Passing

- Unlike the shared memory model, resources are local;
- Each process operates in its own environment (logical address space) and communication occurs via the exchange of messages;
- Messages can be instructions, data or synchronisation signals;
- The message passing scheme can also be implemented on shared memory architectures;
- Delays are much longer than those due to shared variables in the same memory space;

Advantages and Drawbacks

- **Advantages**

- Communications hardware and software are important components of HPC system and often very highly optimised;
- Portable and scalable;
- Long history (many applications already written for it);

- **Drawbacks**

- Explicit nature of message-passing is error-prone and discourages frequent communications;
- Most serial programs need to be completely re-written;
- High memory overheads.

A brief history of MPI

- Before the 1990's, programmers weren't as lucky as us. Many libraries could facilitate building parallel applications, but there was not a standard accepted way of doing it.
- Since most libraries at this time used the same message passing model with only minor feature differences among them, the authors of the libraries and others came together at the Supercomputing 1992 conference to define a standard interface for performing message passing - the Message Passing Interface.
- This standard interface would allow programmers to write parallel applications that were portable to all major parallel architectures.

A brief history of MPI

- By 1994, a complete interface and standard was defined (MPI-1).
Keep in mind that MPI is *only* a definition for an interface, as OpenMP.
- The first implementation has been made available in 1995.
- MPI Forum <https://www.mpi-forum.org/>
- The standards <https://www.mpi-forum.org/docs/>
- Another official reference
<https://www.mcs.anl.gov/research/projects/mpi/>
- A collection of tutorials <https://mpitutorial.com/tutorials/>



The most important concept in message passing is...

...to minimize message passing as much as possible!

To maximise performance, the program should spend as little time as possible communicating data or waiting for other processes.

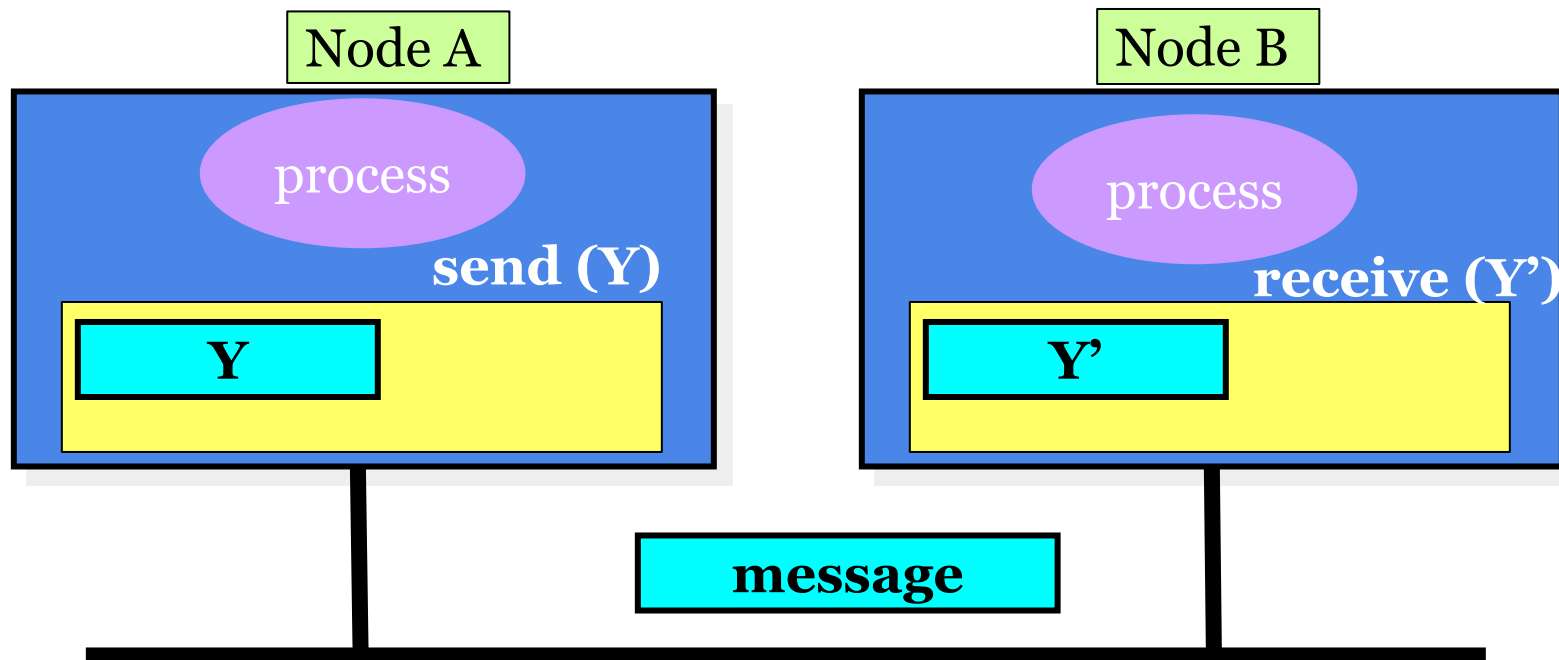
Data transfer and Synchronization

The sender process cooperates with the destination process

The communication system must allow the following three operations:

- `send(message)`
- `receive(message)`
- `synchronization`

MPI Programming Model



Goals of the MPI standard

MPI's prime goals are:

- To allow efficient implementation
- To provide source-code portability

MPI also offers:

- A great deal of functionality
- Support for heterogeneous parallel architectures

MPI2 further extends the library power (parallel I/O, Remote Memory Access, Multi Threads)

MPI3 aims to support exascale by including non-blocking collectives, improved RMA and neighbourhood collectives.

Basic Features of MPI Programs

An MPI program consists of multiple instances of a serial program that communicate by library calls.

Calls may be roughly divided into four classes:

1. Calls used to **initialize, manage, and terminate** communications
2. Calls used to **communicate between pairs** of processors. (point to point communication)
3. Calls used to **communicate among groups** of processors. (collective communication)
4. Calls to create **data types**.

A note about MPI Implementations

- The MPI standard defines the functionalities and the API, i.e. what the C or FORTRAN calls should look like.
- The MPI standard **does not define how the calls should be performed at the system level** (algorithms, buffers, etc) or how the environment is set up (env variables, mpirun or mpiexec, libraries, etc). This is left to the **implementation**.
- There are various implementations (IntelMPI, OpenMPI, MPICH, HPMPI, etc) which have different performances, features and standards compliance.
- On some clusters (e.g. Galileo, Marconi) you may choose which MPI to use, on other systems you have only the vendor-supplied version (IBM MPI for FERMI).

A First Program: Hello World!

Fortran

```
PROGRAM hello

    INCLUDE 'mpif.h'

    INTEGER err

    CALL MPI_INIT(err)

    PRINT *, "hello world!"

    CALL MPI_FINALIZE(err)

END
```

C

```
#include <stdio.h>

#include <mpi.h>

void main (int argc, char * argv[] )
{
    int err;

    err = MPI_Init(&argc, &argv);
    printf("Hello world!\n");
    err = MPI_Finalize();
}
```

Header files

All Subprogram that contains calls to MPI subroutine must include the MPI header file

C:

```
#include <mpi.h>
```

Fortran:

```
include 'mpif.h'
```

Fortran 90:

```
USE MPI
```

Fortran 08 (MPI-3):

```
USE MPI_F08
```

The header file contains definitions of MPI constants, MPI types and functions

FORTTRAN note:

The FORTRAN include and module forms are *not equivalent*: the module can also do type checking. Some compilers gave problems with the module but it is now highly recommended to use the module, particularly for FORTRAN 2008 (most rigorous type-checking)

MPI function format

C:

```
int error = MPI_Xxxxx(parameter,...);  
MPI_Xxxxx(parameter,...);
```

FORTRAN:

```
CALL MPI_XXXXX(parameter, IERROR)  
  
INTEGER IERROR
```

Initializing MPI

C:

```
int MPI_Init(int *argc, char ***argv)
```

FORTRAN:

```
INTEGER IERROR
```

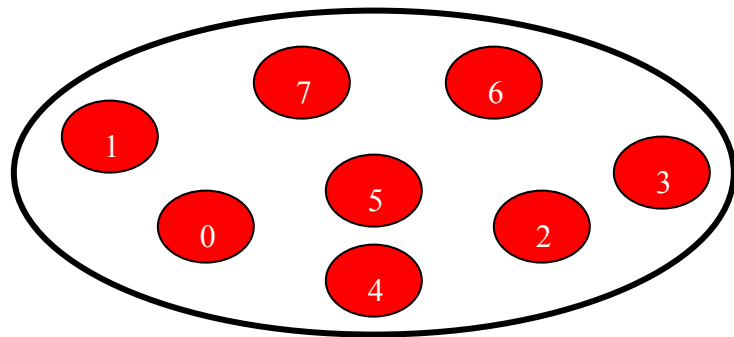
```
MPI_INIT(IERROR)
```

Must be first MPI call: initializes the message passing routines

MPI Communicator

- In MPI it is possible to divide the total number of processes into groups, called *communicators*.
- The Communicator is a variable identifying a group of processes that are allowed to communicate with each other.
- The communicator that includes all processes is called **MPI_COMM_WORLD**
- **MPI_COMM_WORLD** is the default communicator (automatically defined)

All MPI communication subroutines have a communicator argument.
The Programmer can define many communicators at the same time



MPI_COMM_WORLD

Communicator Size

How many processes are associated with a communicator?

C:

```
MPI_Comm_size(MPI_Comm comm, int *size)
```

FORTRAN:

```
INTEGER COMM, SIZE, IERR
```

```
OUTPUT:  SIZE
```

```
CALL MPI_COMM_SIZE(COMM, SIZE, IERR)
```

Process Rank

How can you identify different processes?

What is the ID of a processor in a group?

C:

```
MPI_Comm_rank(MPI_Comm comm, int *rank)
```

Fortran:

```
CALL MPI_COMM_RANK(COMM, RANK, IERR)
```

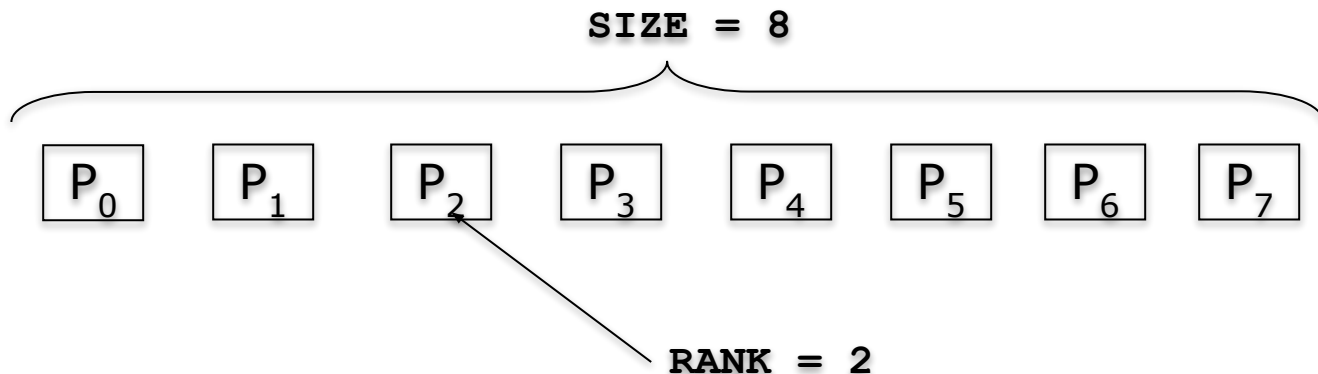
```
INTEGER COMM, RANK, IERR
```

```
OUTPUT:  RANK
```

- *rank* is an integer that identifies the Process inside the communicator *comm*
- MPI_COMM_RANK is used to find the rank (the name or identifier) of the Process running the code

Communicator Size and Process Rank / 1

How many processes are contained within a communicator?



size is the number of processes associated to the communicator

rank is the index of the process within a group associated to a communicator (**rank** = 0,1,...,N-1). The rank is used to identify the source and destination process in a communication

Exiting MPI

Finalizing MPI environment

C:

```
int MPI_Finalize()
```

Fortran:

```
INTEGER IERR
```

```
CALL MPI_FINALIZE(IERR)
```

This two subprograms should be called by all processes, and no other MPI calls are allowed before **mpi_init** and after **mpi_finalize**. Before and after these calls all instances execute the same code (as in serial execution)

MPI_ABORT

- Usage

```
int MPI_Abort( MPI_Comm comm, int errorcode );
```

- Description

Terminates all MPI processes associated with the communicator comm; in most systems (all to date), terminates *all* processes.



A Template for C MPI programs

```
#include <stdio.h>
#include <mpi.h>

void main (int argc, char * argv[])
{
    int err, nproc, myid;

    err = MPI_Init(&argc, &argv);
    err = MPI_Comm_size(MPI_COMM_WORLD, &nproc);
    err = MPI_Comm_rank(MPI_COMM_WORLD, &myid);

    /*** INSERT YOUR PARALLEL CODE HERE ***/

    err = MPI_Finalize();
}
```

Hello, world!

```
/* mpi-hello.c */

#include <mpi.h>
#include <stdio.h>

int main( int argc, char *argv[] )
{
    int rank, size, len;
    char hostname[MPI_MAX_PROCESSOR_NAME];
    MPI_Init( &argc, &argv );
    MPI_Comm_rank( MPI_COMM_WORLD, &rank );
    MPI_Comm_size( MPI_COMM_WORLD, &size );
    MPI_Get_processor_name( hostname, &len );
    printf("Greetings from process %d of %d running on %s\n",
           rank, size, hostname);
    MPI_Finalize();
    return 0;
}
```

No MPI call before this line

No MPI call after this line

Compiling and Running MPI programs

- Implementation is system dependent but it is usual to use the “wrapped” version of the compiler to include the MPI headers and link in the MPI libraries. Wrapped compilers tend to be called `mpif90`, `mpicc`, `mpic++`, etc.
- On HPC systems MPI programs are run via the batch system with appropriate settings. For debugging sometimes it is possible to open interactive sessions (e.g. SLURM on GALILEO).
- a program such as **`mpirun`** or **`mpiexec`** is then used to launch multiple instances of the program on the assigned nodes. For MARCONI and GALILEO you can also use the **`srun`** command.

Usage

COMPILE

- GCC-based MPI `mpicc hello.c -o hello`
- INTEL-based MPI `mpiicc hello.c -o hello`

RUN

Here you decide how many processes will execute the computation

- `mpirun / mpiexec -np 100 ./hello`
- The path is important!
- IN this case you execute all the 100 processes on `hpcocapie01`

<https://software.intel.com/en-us/mpi-developer-reference-linux-introduction>
<https://software.intel.com/en-us/mpi-developer-reference-linux-global-options>

Usage

`mpiexec -nolocal -hostfile list.txt -perhost 1 -np 11 /bin/hostname`

```
[[dadagostino@hpcocapie01 mpi]$ cat list.txt
```

```
hpcocapie01  
hpcocapie02  
hpcocapie03  
hpcocapie04  
hpcocapie05  
hpcocapie06  
hpcocapie07  
hpcocapie08  
hpcocapie09  
hpcocapie10  
hpcocapie11
```

You can choose the file name

2 on hpcocapie02

```
hpcocapie02.ge.infn.it  
hpcocapie02.ge.infn.it  
hpcocapie05.ge.infn.it  
hpcocapie07.ge.infn.it  
hpcocapie08.ge.infn.it  
hpcocapie03.ge.infn.it  
hpcocapie04.ge.infn.it  
hpcocapie06.ge.infn.it  
hpcocapie09.ge.infn.it  
hpcocapie11.ge.infn.it  
hpcocapie10.ge.infn.it
```

Try

`mpiexec -hostfile list.txt -np 11 /bin/hostname`

`mpiexec -hostfile list.txt -nolocal -np 11 /bin/hostname`

`mpiexec -hostfile list.txt -nolocal -perhost 1 -np 15 /bin/hostname`

Example

```
mpiexec -hostfile list.txt -nolocal -perhost 1 -np 15 ./hello2
```

```
#include <stdio.h>
#include <mpi.h>
void main (int argc, char * argv[])
{
    int err, nproc, myid;
    err = MPI_Init(&argc, &argv);
    err = MPI_Comm_size(MPI_COMM_WORLD, &nproc);
    err = MPI_Comm_rank(MPI_COMM_WORLD, &myid);
    printf("I'm proc %d of total %d\n",myid,nproc);
    err = MPI_Finalize();
}
```

Using bsub - examples

<https://web.ge.infn.it/calcolo/joomla/2-uncategorised/106-farm-hpc-ocapie#h4-sottomissione-di-job>

(ITA, manual at the end of the page). You'll receive results on the ge.infn.it mail address

```
bsub -q ocapie -n 30 mpiexec -np 30 /bin/hostname
```

All on a single node

```
bsub -q ocapie -R span[ptile=1] -n 30 mpiexec -np 30 /bin/hostname
```

nodes

Never executed, you should have 30

```
bjobs 155735
```

JOBID	USER	STAT	QUEUE	FROM_HOST	EXEC_HOST	JOB_NAME	SUBMIT_TIME
155735	dadagos	PEND	ocapie	hpcocapie01	*/hostname		Nov 8 17:03

```
bsub -q ocapie -R span[ptile=5] -n 30 mpiexec -np 30 /bin/hostname
```

Job was executed on host(s) <5*hpcocapie07.ge.infn.it>, <5*hpcocapie01.ge.infn.it>, <5*hpcocapie02.ge.infn.it>
<5*hpcocapie05.ge.infn.it>, <5*hpcocapie08.ge.infn.it>, <5*hpcocapie09.ge.infn.it>

```
bsub -q ocapie -n 300 mpiexec -np 300 /bin/hostname
```

Job was executed on host(s) <256*hpcocapie01.ge.infn.it>, <44*hpcocapie07.ge.infn.it>

SPMD

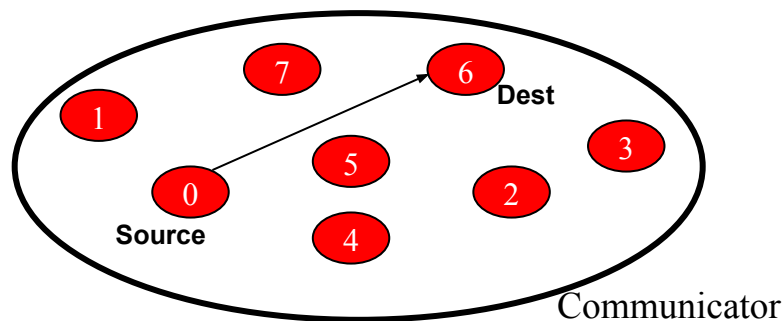
(Single Program Multiple Data)

- The **same** program is executed by P processes
- Each process may choose a different execution path depending on its ID (*rank*)

```
...
MPI_Init(...);
foo();                               /* executed by all processes */
if ( my_id == 0 ) {
    do_something();                 /* executed by process 0 only */
} else {
    do_something_else();           /* executed by all other processes */
}
MPI_Finalize();
...
```

Point-to-Point Communication

- It is the basic communication method provided by MPI library. Communication between 2 processes
- It is conceptually simple: source process A sends a message to destination process B, B receive the message from A.
- Communication take places within a **communicator**
- Source and Destination are identified by their rank in the communicator



Point-to-Point communication quick example

The construction

```
if rank equals i
```

```
send information
```

```
else if rank equals j
```

```
receive information
```

is very common in MPI programs. Often one rank (usually rank 0) is selected for particular tasks which can be or should be done by one task only such as reading or writing files, giving messages to the user or for managing the overall logic of the program (e.g. master-slave).

A Simple MPI Program

```
/* mpi-point-to-point.c */
#include <mpi.h>
#include <stdio.h>
int main( int argc, char *argv[])
{
    int rank, buf;
    MPI_Status status;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    /* process 0 sends and process 1 receives */
    if (rank == 0) {
        buf = 123456;
        MPI_Send(&buf, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    } else if (rank == 1) {
        MPI_Recv(&buf, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &status);
        printf("Received %d\n", buf);
    }

    MPI_Finalize();
    return 0;
}
```

MPI Tags

- Messages are sent with an accompanying user-defined integer *tag*, to assist the receiving process in identifying the message
- Messages can be screened at the receiving end by specifying a specific tag, or not screened by specifying **MPI_ANY_TAG** as the tag in a receive

The Message

- **Buffers** of data are exchanged. They are series of **count** elements of a particular MPI **data type**
- In this way MPI knows how many bytes (length) has to send/receive.
- This allows MPI programs to run in **heterogeneous** environments
- C types are different from Fortran types.

Messages are identified by their envelopes. A message could be exchanged only if the sender and receiver specify the correct envelope

Message Structure

envelope				body		
source	destination	communicator	tag	buffer	count	datatype

Data Types

- MPI Data types
 - Basic types (portability)
 - Derived types (MPI_Type_xxx functions)
- Derived type can be built up from basic types
- User-defined data types allows MPI to use non-contiguous buffers of data
- MPI defines '*handles*' to allow programmers to refer to data types
 - declare the **right type** of the handles as defined in the API of language (C/Fortran)
 - MPI data type and language data type have to be **compatible** (MPI_INTEGER and INTEGER)

C - MPI Intrinsic Datatypes

MPI Data type	C Data type
<code>MPI_CHAR</code>	<code>signed char</code>
<code>MPI_SHORT</code>	<code>signed short int</code>
<code>MPI_INT</code>	<code>signed int</code>
<code>MPI_LONG</code>	<code>signed long int</code>
<code>MPI_UNSIGNED_CHAR</code>	<code>unsigned char</code>
<code>MPI_UNSIGNED_SHORT</code>	<code>unsigned short int</code>
<code>MPI_UNSIGNED</code>	<code>unsigned int</code>
<code>MPI_UNSIGNED_LONG</code>	<code>unsigned long int</code>
<code>MPI_FLOAT</code>	<code>float</code>
<code>MPI_DOUBLE</code>	<code>double</code>
<code>MPI_LONG_DOUBLE</code>	<code>long double</code>
<code>MPI_BYTE</code>	https://www.mcs.anl.gov/research/projects/mpi/mpi-standard/mpi-report-1.1/node32.htm
<code>MPI_PACKED</code>	

For a communication to succeed ...

1. Sender must specify a valid destination rank.
2. Receiver must specify a valid source rank.
3. The communicator must be the same.
4. Tags must match.
5. Buffers must be large enough.

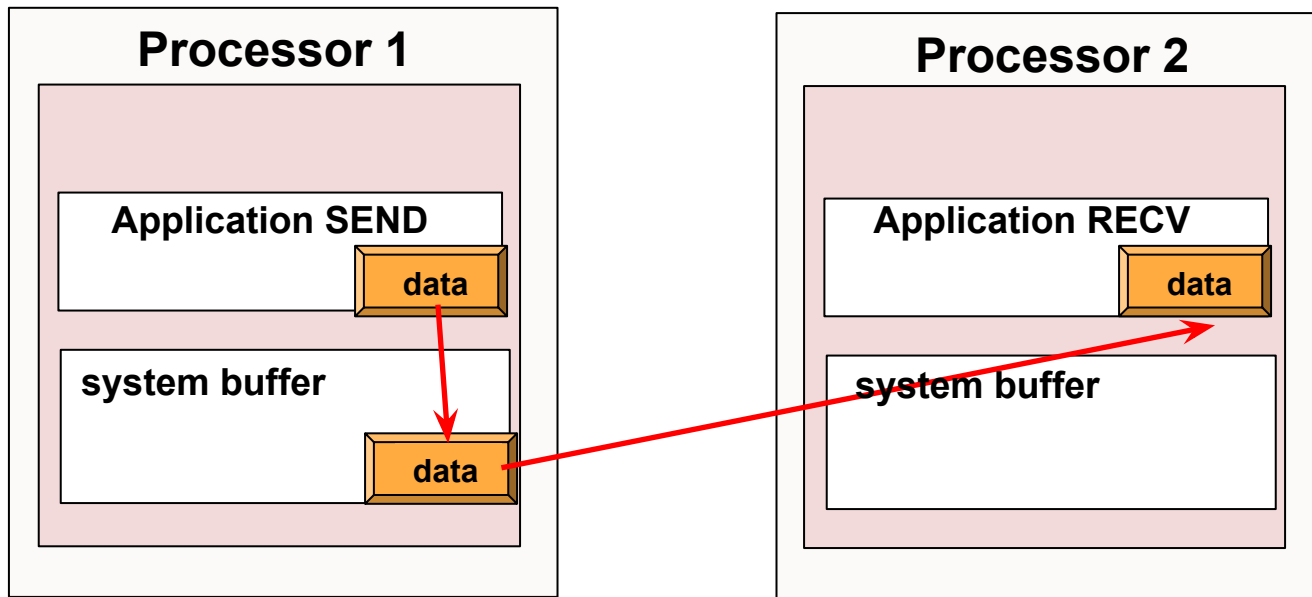
Completion

- In a perfect world, every send operation would be perfectly synchronized with its matching receive. This is rarely the case. The MPI implementation is able to deal with the exchanging data when the two tasks are out of sync.
- **Completion** of the communication means that memory locations used in the message transfer can be safely accessed
 - Send: variable sent can be reused after completion
 - Receive: variable received can be used after completion

Blocking communications

- Most of the MPI point-to-point routines can be used in either **blocking or non-blocking** mode.
- Blocking:
 - A blocking **send** returns **after it is safe to modify the sent buffer**. Safe does not imply that the data was actually received - it may be stored in a system buffer.
 - A blocking **send** can be **synchronous**.
 - A blocking **send** can be **asynchronous** if a system **buffer** is used to hold the data for eventual delivery.
 - A blocking **receive** only "returns" after the data **has arrived and is ready for use by the program**.

Blocking Communications



Standard Send and Receive

C:

```
int MPI_Send(void *buf, int count, MPI_Datatype type,  
             int dest, int tag, MPI_Comm comm);
```

```
int MPI_Recv (void *buf, int count, MPI_Datatype type,  
             int source, int tag, MPI_Comm comm, MPI_Status *status);
```

Standard Send and Receive

Basic blocking point-to-point communication routine in MPI.

Fortran:

`MPI_SEND(buf, count, type, dest, tag, comm, ierr)`

`MPI_RECV(buf, count, type, source, tag, comm, status, ierr)`

Message body

Message envelope

buf	send/receive buffer (memory located in sender/receiver memory).
count	(INTEGER) number of contiguous elements of buf to be sent
type	(INTEGER) MPI type of buf (implicitly number of bytes)
dest	(INTEGER) rank of the destination process
tag	(INTEGER) number identifying the message
comm	(INTEGER) communicator of the sender and receiver
status	(INTEGER) array of size MPI_STATUS_SIZE containing communication status information (Orig Rank, Tag, Number of elements received)
ierr	(INTEGER) error code (if ierr=0 no error occurs)

Send and Receive - C

```
#include <stdio.h>
#include <mpi.h>

void main (int argc, char * argv[])
{
    int err, nproc, myid;
    MPI_Status status;
    float a[2];

    err = MPI_Init(&argc, &argv);
    err = MPI_Comm_size(MPI_COMM_WORLD, &nproc);
    err = MPI_Comm_rank(MPI_COMM_WORLD, &myid);

    if( myid == 0 ) {
        a[0] = 3.0, a[1] = 5.0;
        MPI_Send(a, 2, MPI_FLOAT, 1, 10, MPI_COMM_WORLD);
    } else if( myid == 1 ) {
        MPI_Recv(a, 2, MPI_FLOAT, 0, 10, MPI_COMM_WORLD, &status);
        printf("%d: a[0]=%f a[1]=%f\n", myid, a[0], a[1]);
    }

    err = MPI_Finalize();
}
```

MPI_Status

- **MPI_Status** is a C structure with the following fields:
 - `int MPI_SOURCE;`
 - `int MPI_TAG;`
 - `int MPI_ERROR;`
- Therefore, a process can check the actual source and tag of a message received with **MPI_ANY_TAG** or **MPI_ANY_SOURCE**

MPI_Get_count()

```
MPI_Get_count( &status, datatype, &count );
```

- **MPI_Recv** may complete even if less than *count* elements have been received
 - Provided that the matching **MPI_Send** actually sent fewer elements
- **MPI_Get_count** can be used to know how many elements of type *datatype* have actually been received
- See [mpi-get-count.c](#)

Example

```
#include <stdio.h> #include <stdlib.h>
#include <time.h> #include <mpi.h>
#define BUFLLEN 16

int main( int argc, char *argv[]) {
    int rank, buf[BUFLLEN] = {0};
    int count, i;
    MPI_Status status;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank( MPI_COMM_WORLD, &rank );

    /* Process 0 sends, Process 1 receives */
    if (rank == 0) {
        srand(time(NULL));
        /* Fills the buffer with a random number of integers */
        count = 1 + rand()%BUFLLEN;
        for (i=0; i<count; i++) { buf[i] = i; }
        MPI_Send(buf, count, MPI_INT, 1, 0, MPI_COMM_WORLD);
        printf("Sent %d integers\n", count);
    }
```

```
else if (rank == 1) {
    MPI_Recv(buf, BUFLLEN, MPI_INT, 0, 0, \
MPI_COMM_WORLD, &status );
    MPI_Get_count(&status, MPI_INT, &count);
    printf( "Received %d integers: ", count );
    for (i=0; i<count; i++) { printf("%d ", buf[i]); }
    printf("\n");
}
MPI_Finalize();
return 0;
```

```
}
[[dadagostino@hpcocapie01 mpi]$ mpiexec -np 2 ./mpi-get-count
Sent 12 integers
Received 12 integers: 0 1 2 3 4 5 6 7 8 9 10 11
[[dadagostino@hpcocapie01 mpi]$ mpiexec -np 2 ./mpi-get-count
Sent 16 integers
Received 16 integers: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
[[dadagostino@hpcocapie01 mpi]$ mpiexec -np 2 ./mpi-get-count
Sent 15 integers
Received 15 integers: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14
[[dadagostino@hpcocapie01 mpi]$ mpiexec -np 2 ./mpi-get-count
Sent 15 integers
Received 15 integers: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14
[[dadagostino@hpcocapie01 mpi]$ mpiexec -np 2 ./mpi-get-count
Sent 15 integers
Received 15 integers: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14
```

Non Blocking communications

- Non-blocking **send** and **receive** routines will **return almost immediately**. They do not wait for any communication events to complete
- Non-blocking operations simply "request" the MPI library to perform the operation **when it is possible**. The user can not predict when that will happen.
- It is unsafe to modify the application buffer until you know for a fact that the requested non-blocking operation was actually performed by the library. There are **"wait"** routines used to do this.
- Non-blocking communications are primarily used to **overlap computation with communication**.
- Another important use is to **overlap communication with communication**, i.e. avoiding serialization issues (for example, when a loop of communications is involved)

Non-Blocking Send and Receive

C:

```
int MPI_Isend(void *buf, int count, MPI_Datatype type,  
             int dest, int tag, MPI_Comm comm, MPI_Request *req);
```

```
int MPI_Irecv (void *buf, int count, MPI_Datatype type,  
             int source, int tag, MPI_Comm comm, MPI_Request *req);
```

Non-Blocking Send and Receive

FORTRAN:

```
MPI_ISEND(buf, count, type, dest, tag, comm, req, ierr)
```

```
MPI_IRecv(buf, count, type, source, tag, comm, req, ierr)
```

buf	send/receive buffer (memory located in sender/receiver memory).
count	(INTEGER) number of contiguous elements of buf to be sent
type	(INTEGER) MPI type of buf
dest	(INTEGER) rank of the destination process
tag	(INTEGER) number identifying the message
comm	(INTEGER) communicator of the sender and receiver
req	(INTEGER) output, identifier of the communications handle
ierr	(INTEGER) output, error code (if ierr=0 no error occurs)

Waiting for Completion

FORTRAN:

```
MPI_WAIT(req, status, ierr)
```

```
MPI_WAITALL (count, array_of_requests, array_of_statuses, ierr)
```

A call to this subroutine causes the code to wait until the communication referred to by `req` is complete.

`req` (INTEGER): input/output, identifier associated to a communications event (initiated by **MPI_ISEND** or **MPI_IRECV**).

`Status` (INTEGER): array of size **MPI_STATUS_SIZE**, if `req` was associated to a call to **MPI_IRECV**, `status` contains informations on the received message, otherwise `status` could contain an error code.

`ierr` (INTEGER): output, error code (if `ierr=0` no error occurs).

C:

```
int MPI_Wait(MPI_Request *req, MPI_Status *status);
```

```
Int MPI_Waitall (count, &array_of_requests, &array_of_statuses) ;
```

Testing Completion

FORTRAN:

```
MPI_TEST(req, flag, status, ierr)
```

```
MPI_TESTALL (count,array_of_requests,flag,array_of_statuses,ierr)
```

A call to this subroutine sets **flag** to **.true.** if the communication pointed by **req** is complete, sets **flag** to **.false.** otherwise.

Req (INTEGER) input/output, identifier associated to a communications event (initiated by **MPI_ISEND** or **MPI_IRECV**).

Flag (LOGICAL) output, **.true.** if communication **req** has completed **.false.** otherwise

Status (INTEGER) array of size MPI_STATUS_SIZE, if req was associated to a call to **MPI_IRECV**, **status** contains informations on the received message, otherwise **status** could contain an error code.

ierr (INTEGER) output, error code (if **ierr=0** no error occurs).

C:

```
int MPI_Test (&request,&flag,&status) ;
```

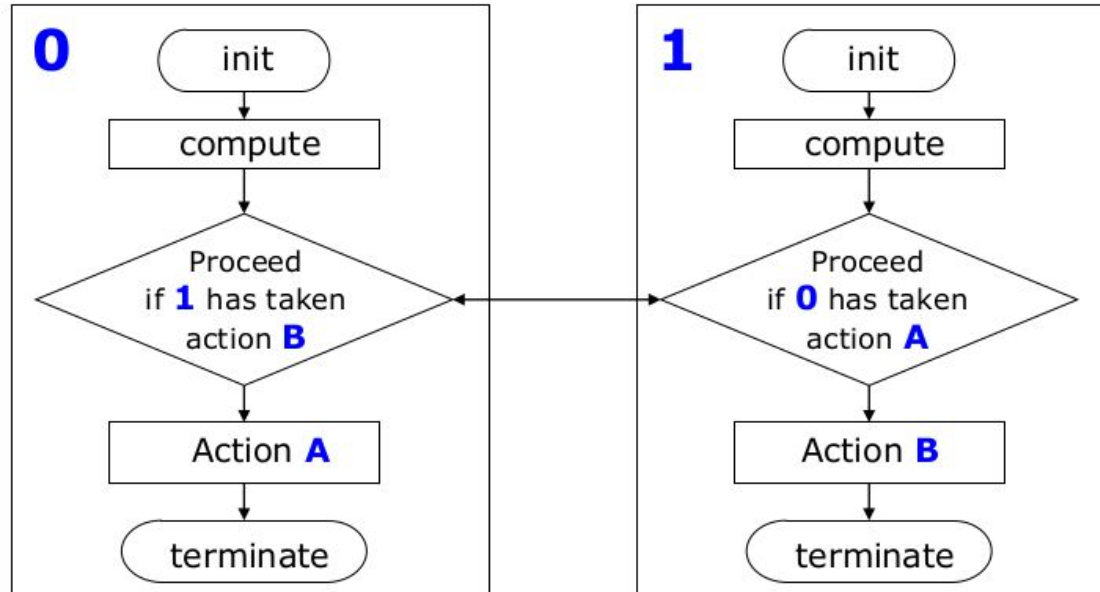
```
Int MPI_Testall (count,&array_of_requests,&flag,&array_of_statuses) ;
```

Wildcards

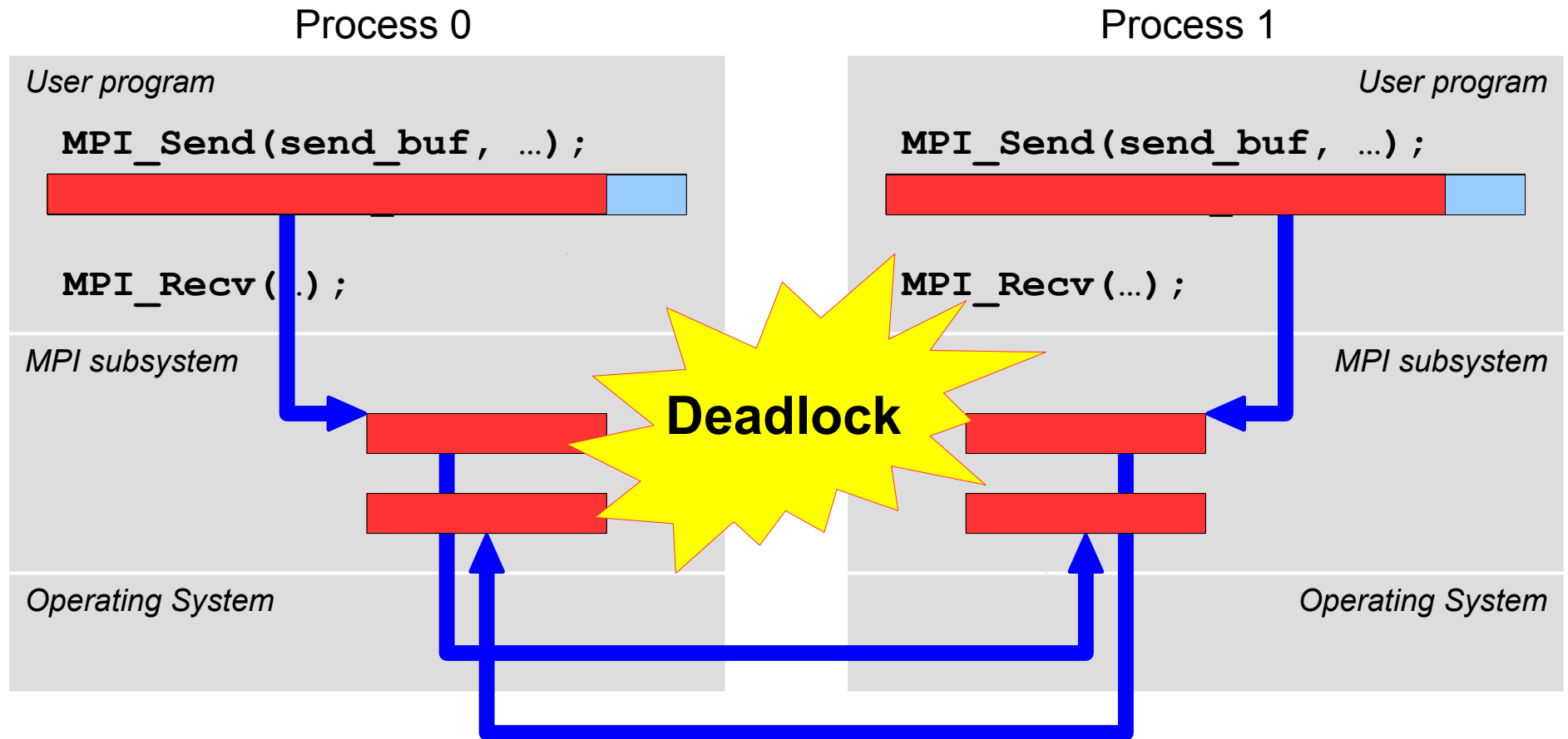
- Both in FORTRAN and C **MPI_RECV** accepts wildcards:
- To receive from any source: **MPI_ANY_SOURCE**
- To receive with any tag: **MPI_ANY_TAG**
- Actual source and tag are returned in the receiver's status parameter

DEADLOCK

A Deadlock occurs when 2 (or more) processes are blocked and each one is waiting for the other to make progress.



Blocking communication and deadlocks



Simple DEADLOCK

```

PROGRAM deadlock
  USE mpi
  implicit none
  INTEGER ierr, myid, nproc
  INTEGER status(MPI_STATUS_SIZE)
  REAL A(2), B(2)

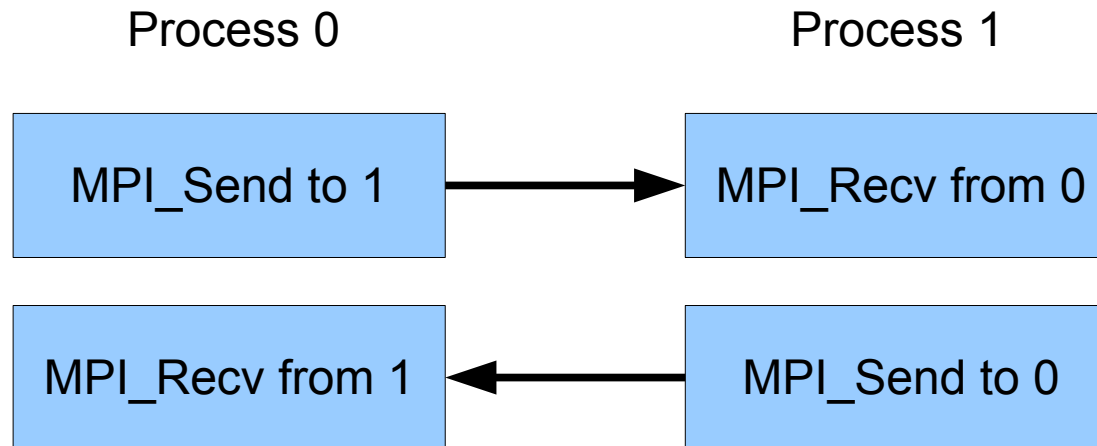
  CALL MPI_INIT(ierr)
  CALL MPI_COMM_SIZE(MPI_COMM_WORLD, nproc, ierr)
  CALL MPI_COMM_RANK(MPI_COMM_WORLD, myid, ierr)

  IF( myid .EQ. 0 ) THEN
    a(1) = 2.0
    a(2) = 4.0
    CALL MPI_SEND(a, 2, MPI_REAL, 1, 10, MPI_COMM_WORLD, ierr)
    CALL MPI_RECV(b, 2, MPI_REAL, 1, 11, MPI_COMM_WORLD, status, ierr)
  ELSE IF( myid .EQ. 1 ) THEN
    a(1) = 3.0
    a(2) = 5.0
    CALL MPI_SEND(a, 2, MPI_REAL, 0, 11, MPI_COMM_WORLD, ierr)
    CALL MPI_RECV(b, 2, MPI_REAL, 0, 10, MPI_COMM_WORLD, status, ierr)
  END IF
  WRITE(6,*) myid, ': b(1)=', b(1), ' b(2)=', b(2)
  CALL MPI_FINALIZE(ierr)
END PROGRAM

```

Blocking communication and deadlocks

- To avoid the deadlock it is necessary to reorder the operations so that send/receive pairs match...



- ...or use **non-blocking** communication primitives
... or smart implementations as the Intel one.

Avoiding DEADLOCK

```

PROGRAM avoid_lock
  USE mpi
  Implicit none
  INTEGER ierr, myid, nproc
  INTEGER status(MPI_STATUS_SIZE)
  REAL A(2), B(2)

  CALL MPI_INIT(ierr)
  CALL MPI_COMM_SIZE(MPI_COMM_WORLD, nproc, ierr)
  CALL MPI_COMM_RANK(MPI_COMM_WORLD, myid, ierr)

  IF( myid .EQ. 0 ) THEN
    a(1) = 2.0
    a(2) = 4.0
    CALL MPI_RECV(b, 2, MPI_REAL, 1, 11, MPI_COMM_WORLD, status, ierr)
    CALL MPI_SEND(a, 2, MPI_REAL, 1, 10, MPI_COMM_WORLD, ierr)
  ELSE IF( myid .EQ. 1 ) THEN
    a(1) = 3.0
    a(2) = 5.0
    CALL MPI_SEND(a, 2, MPI_REAL, 0, 11, MPI_COMM_WORLD, ierr)
    CALL MPI_RECV(b, 2, MPI_REAL, 0, 10, MPI_COMM_WORLD, status, ierr)
  END IF
  WRITE(6,*) myid, ' : b(1)=', b(1), ' b(2)=', b(2)
  CALL MPI_FINALIZE(ierr)
END PROGRAM

```

Avoiding DEADLOCK (2)

PROGRAM deadlock

```

USE mpi
implicit none
INTEGER ierr, myid, nproc, req
INTEGER status(MPI_STATUS_SIZE)
REAL A(2), B(2)

CALL MPI_INIT(ierr)
CALL MPI_COMM_SIZE(MPI_COMM_WORLD, nproc, ierr)
CALL MPI_COMM_RANK(MPI_COMM_WORLD, myid, ierr)

IF( myid .EQ. 0 ) THEN
    a(1) = 2.0
    a(2) = 4.0
    CALL MPI_ISEND(a, 2, MPI_REAL, 1, 10, MPI_COMM_WORLD, req, ierr)
    CALL MPI_RECV(b, 2, MPI_REAL, 1, 11, MPI_COMM_WORLD, status, ierr)
    CALL MPI_WAIT(req, status, ierr)
ELSE IF( myid .EQ. 1 ) THEN
    a(1) = 3.0
    a(2) = 5.0
    CALL MPI_SEND(a, 2, MPI_REAL, 0, 11, MPI_COMM_WORLD, ierr)
    CALL MPI_RECV(b, 2, MPI_REAL, 0, 10, MPI_COMM_WORLD, status, ierr)
END IF
WRITE(6,*) myid, ': b(1)=', b(1), ' b(2)=', b(2)
CALL MPI_FINALIZE(ierr)
END PROGRAM

```

SendRecv

- Send a message and post a receive before blocking. Will block until the sending application buffer is free for reuse and until the receiving application buffer contains the received message.
- The easiest way to send and receive data without worrying about deadlocks

FORTRAN:

Sender side

```
CALL MPI_SENDRECV(sndbuf, snd_size, snd_type, destid, tag,  
rcvbuf, rcv_size, rcv_type, sourceid, tag, comm, status,  
ierr)
```

**Receiver
side**

SendRecv example

```
#include <mpi.h>
#include <stdio.h>

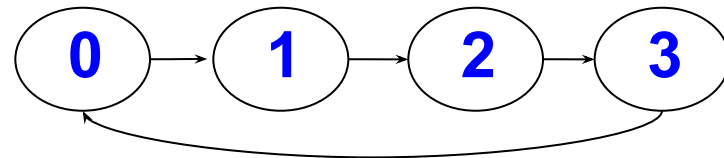
int main(int argc, char *argv[])
{
    int myid, numprocs, left, right;
    int buffer[1], buffer2[1];
    MPI_Status status;

    MPI_Init(&argc,&argv);
    MPI_Comm_size(MPI_COMM_WORLD, &numprocs);
    MPI_Comm_rank(MPI_COMM_WORLD, &myid);

    right = (myid + 1) % numprocs;
    left = myid - 1;
    if (left < 0)
        left = numprocs - 1;
    buffer[0]=myid;
    MPI_Sendrecv(buffer, 1, MPI_INT, right, 123, buffer2, 1, MPI_INT, left, 123,
        MPI_COMM_WORLD, &status);

    printf("I am rank %d and I have received %d\n",myid,buffer2[0]);
    MPI_Finalize();

    return 0;
}
```



Useful for cyclic
communication
patterns

SEND and RECV variants

Mode	Completion Condition	Blocking subroutine	Non-blocking subroutine
Standard send	Message sent (receive state unknown)	<code>MPI_SEND</code>	<code>MPI_ISEND</code>
receive	Completes when a matching message has arrived	<code>MPI_RECV</code>	<code>MPI_Irecv</code>
Synchronous send	Only completes after a matching <code>recv()</code> is posted and the receive operation is started.	<code>MPI_SSEND</code>	<code>MPI_ISSEND</code>
Buffered send	Always completes, irrespective of receiver Guarantees the message being buffered	<code>MPI_BSEND</code>	<code>MPI_IBSEND</code>
Ready send	Always completes, irrespective of whether the receive has completed	<code>MPI_RSEND</code>	<code>MPI_IRSEND</code>

Final Comments

- MPI is a standard for message-passing and has numerous implementations (OpenMPI, IntelMPI, MPICH, etc)
- MPI uses send and receive calls to manage communications between two processes (point-to-point)
- The calls can be blocking or non-blocking.
- Non-blocking calls can be used to overlap communication with computation (or communication) but wait routines are needed for synchronization.
- Deadlock is a common error and is due to incorrect order of send/receive

Nicola Spallanzani

n.spallanzani@cineca.it

High Performance Computing department

Parallel programming with MPI

Collective Communications

www.cineca.it



Collective communications

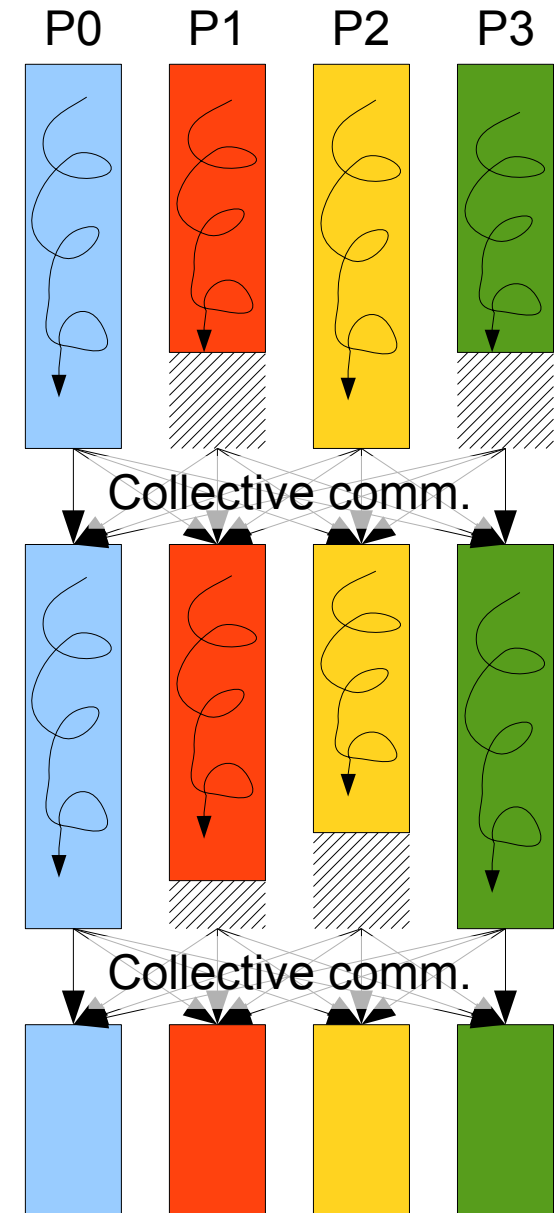
Collective communications is a method of communication which involves all processes in a communicator:

- All processes (in a communicator) call the collective function
- Collective communications will not interfere with point-to-point
- All collective communications are blocking (in MPI 2.0)
- No tags are required
- Receive buffers must match in size (number of bytes)

It's a safe communication mode

Collective communications

- Many applications use the *bulk synchronous* pattern:
 - Repeat:
 - Local computation
 - Communicate to update global view on all processes
- Collective communications are executed by all processes in the group to compute and share some global result



Collective communications

- Collective communications are assumed to be more efficient than point-to-point operations achieving the same result
- Understanding when collective communications are to be used is an **essential skill** of a MPI programmer

Collective communications

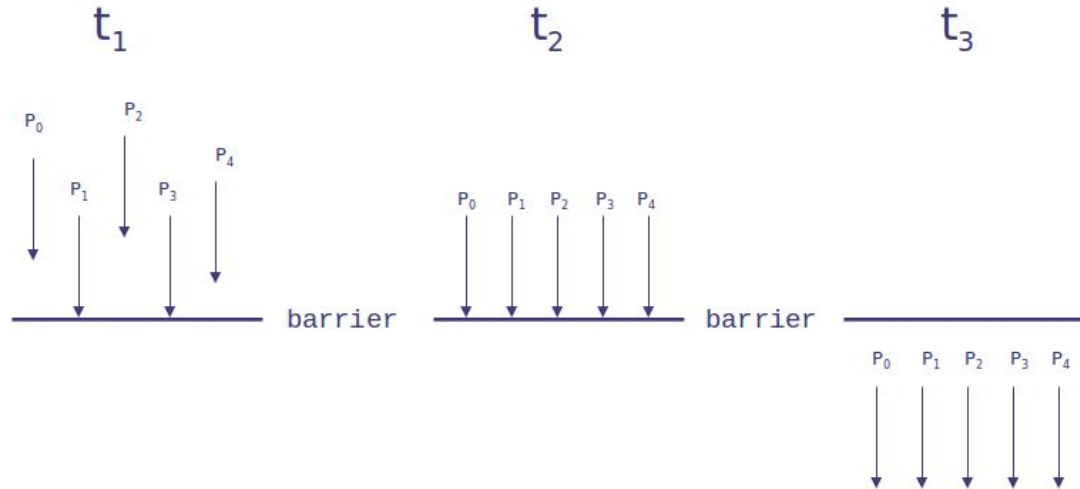
Communications involving a group of processes. They are called by all the ranks involved in a communicator (or a group) and are of three types:

- Synchronization (e.g. Barrier)
- Data Movement (e.g. Broadcast or Gather/scatter)
- Global Computation (e.g. reductions)

MPI Barrier

It stops all processes within a communicator until they are synchronized

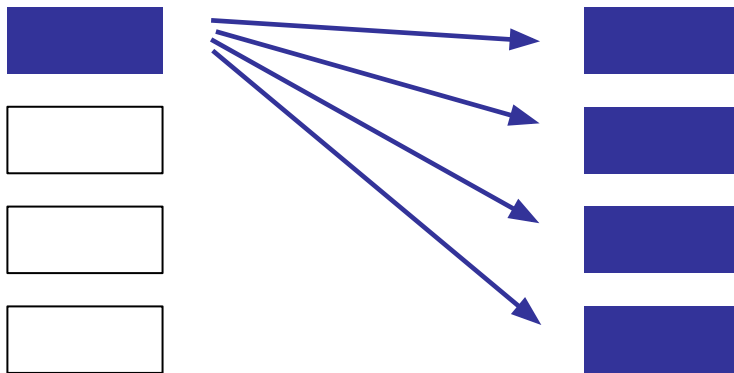
```
int MPI_Barrier(MPI_Comm comm);
```



MPI Broadcast

```
int MPI_Bcast (void *buf, int count, MPI_Datatype datatype,  
               int root, MPI_Comm comm);
```

Note that all processes must specify the same root and same comm.



MPI_Bcast()

Broadcasts a message to all other processes of a group

```
count = 3;  
src = 1; /* broadcast originates from process 1 */  
MPI_Bcast(buf, count, MPI_INT, src, MPI_COMM_WORLD);
```

Proc 0

Proc 1

Proc 2

Proc 3



`buf[]` (before)



`buf[]` (after)

Example

```
PROGRAM broad_cast
  INCLUDE 'mpif.h'
  INTEGER ierr, myid, nproc, root
  INTEGER status(MPI_STATUS_SIZE)
  REAL A(2)
  CALL MPI_INIT(ierr)
  CALL MPI_COMM_SIZE(MPI_COMM_WORLD, nproc, ierr)
  CALL MPI_COMM_RANK(MPI_COMM_WORLD, myid, ierr)
  root = 0
  IF( myid .EQ. root ) THEN
    a(1) = 2.0
    a(2) = 4.0
  END IF
  CALL MPI_BCAST(a, 2, MPI_REAL, root, MPI_COMM_WORLD, ierr)
  WRITE(6,*) myid, ': a(1)=', a(1), ' a(2)=', a(2)
  CALL MPI_FINALIZE(ierr)
END PROGRAM broad_cast
```

MPI Gather

Each process, root included, sends the content of its send buffer to the root process. The root process receives the messages and stores them in the rank order.

```
int MPI_Gather(void *sendbuf, int sendcnt, MPI_Datatype sendtype,
               void *recvbuf, int recvcnt, MPI_Datatype recvtype,
               int root, MPI_Comm comm);
```



MPI_Gather()

Gathers together data from other processes

```
sendcnt = 3; /* how many items are sent by each process */
recvcnt = 3; /* how many items are received from each process */
dst = 1;     /* message will be gathered at process 1 */
MPI_Gather(sendbuf, sendcnt, MPI_INT,
            recvbuf, recvcnt, MPI_INT, dst, MPI_COMM_WORLD);
```

sendbuf[] (before)

Proc 0	1	2	3
Proc 1	4	5	6
Proc 2	7	8	9
Proc 3	10	11	12

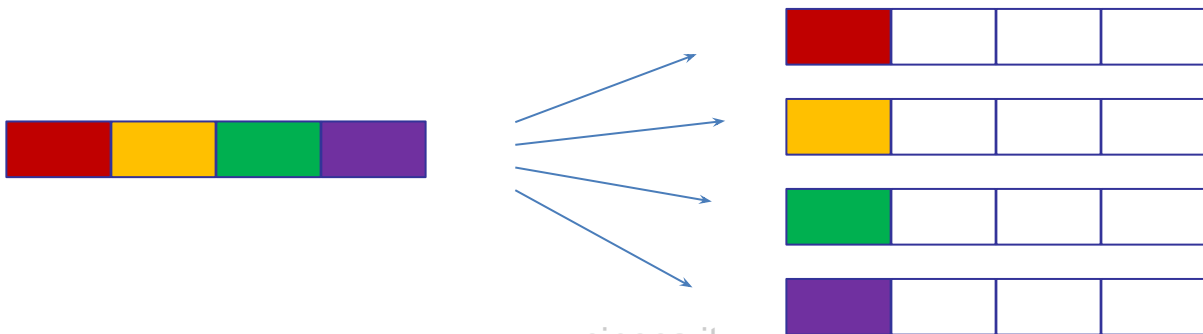
recvbuf[] (after)

1	2	3	4	5	6	7	8	9	10	11	12
---	---	---	---	---	---	---	---	---	----	----	----

MPI Scatter

The root sends a message. The message is split into n equal segments, the i -th segment is sent to the i -th process in the group and each process receives this message.

```
int MPI_Scatter(void *sendbuf, int sendcnt, MPI_Datatype sendtype,  
               void *recvbuf, int recvcnt, MPI_Datatype recvtype,  
               int root, MPI_Comm comm);
```



MPI_Scatter()

Distribute data to other processes in a group

```
sendcnt = 3; /* how many items are sent to each process */
recvcnt = 3; /* how many items are received by each process */
src = 1; /* process 1 contains the message to be scattered */
MPI_Scatter(sendbuf, sendcnt, MPI_INT,
             recvbuf, recvcnt, MPI_INT, src, MPI_COMM_WORLD);
```

sendbuf[] (before)

recvbuf[] (after)

Proc 0

1	2	3
---	---	---

Proc 1

1	2	3	4	5	6	7	8	9	10	11	12
---	---	---	---	---	---	---	---	---	----	----	----

4	5	6
---	---	---

Proc 2

7	8	9
---	---	---

Proc 3

10	11	12
----	----	----

MPI_Scatter()

- The **MPI_Scatter()** operation produce the same result as if the root executes a series of

```
MPI_Send(sendbuf + i * sendcount * extent(sendtype),  
sendcount, sendtype, i, ...)
```

and all other processes execute

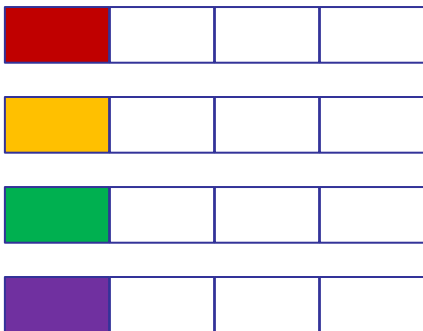
```
MPI_Recv(recvbuf, recvcount, recvtype, i, ...)
```

MPI Allgather

There are possible combinations of collective functions.

For example, MPI Allgather is a combination of a gather + a broadcast:

```
int MPI_Allgather(void *sendbuf, int sendcount, MPI_Datatype sendtype,
                 void *recvbuf, int recvcount, MPI_Datatype recvtype,
                 MPI_Comm comm);
```



MPI_Allgather()

Gathers data from other processes and distribute to all

```
sendcnt = 3;  
recvcnt = 3;  
MPI_Allgather(sendbuf, sendcnt, MPI_INT,  
               recvbuf, recvcnt, MPI_INT, MPI_COMM_WORLD);
```

sendbuf[] (before)

Proc 0	1	2	3
Proc 1	4	5	6
Proc 2	7	8	9
Proc 3	10	11	12

recvbuf[] (after)

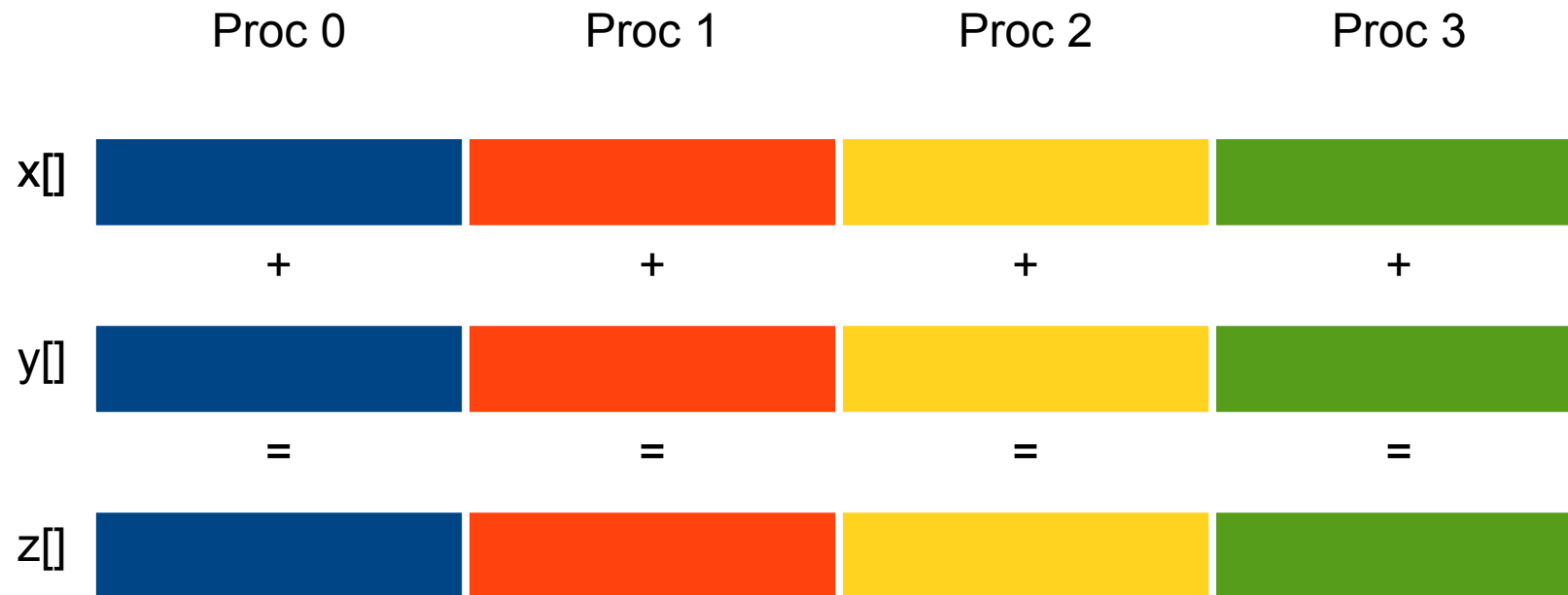
1	2	3	4	5	6	7	8	9	10	11	12
1	2	3	4	5	6	7	8	9	10	11	12
1	2	3	4	5	6	7	8	9	10	11	12
1	2	3	4	5	6	7	8	9	10	11	12

Example: Parallel Vector Sum

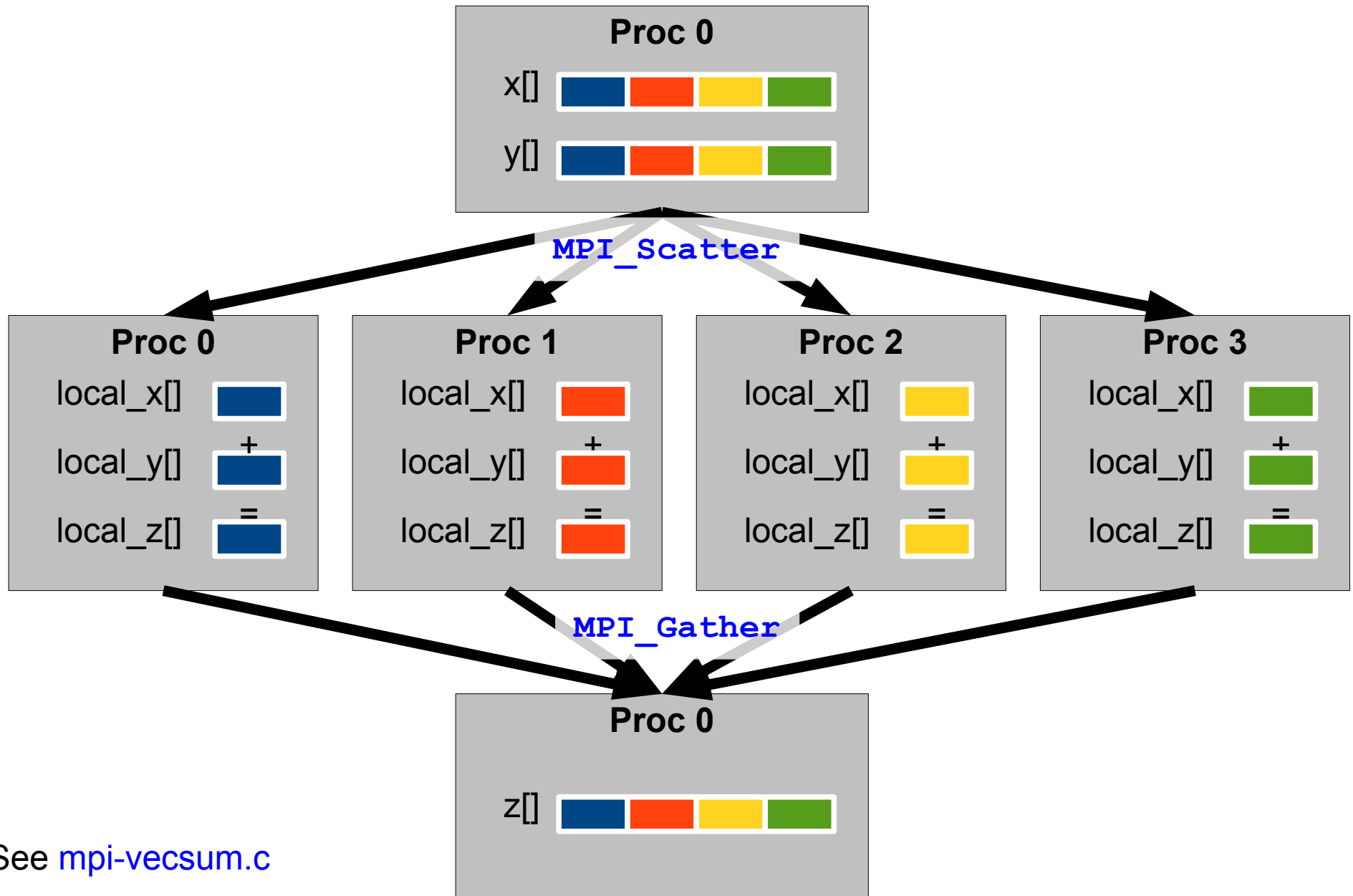
$$\begin{aligned}\mathbf{x} + \mathbf{y} &= (x_0, x_1, \dots, x_{n-1}) + (y_0, y_1, \dots, y_{n-1}) \\ &= (x_0 + y_0, x_1 + y_1, \dots, x_{n-1} + y_{n-1}) \\ &= (z_0, z_1, \dots, z_{n-1}) \\ &= \mathbf{z}\end{aligned}$$

```
void sum( double* x, double* y, double* z, int n )
{
    int i;
    for (i=0; i<n; i++) {
        z[i] = x[i] + y[i];
    }
}
```

Parallel Vector Sum



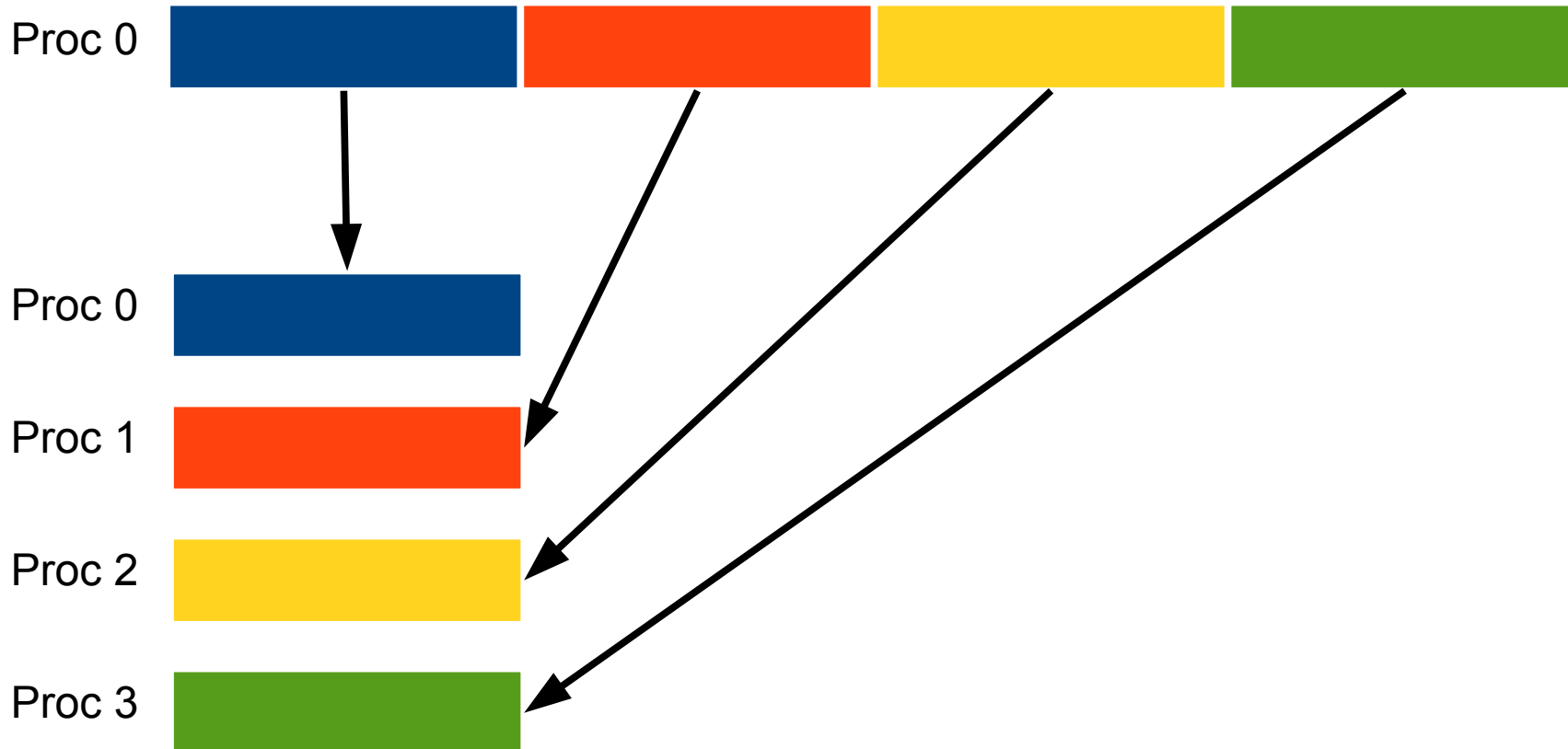
Parallel Vector Sum



See [mpi-vecsum.c](#)

MPI_Scatter()

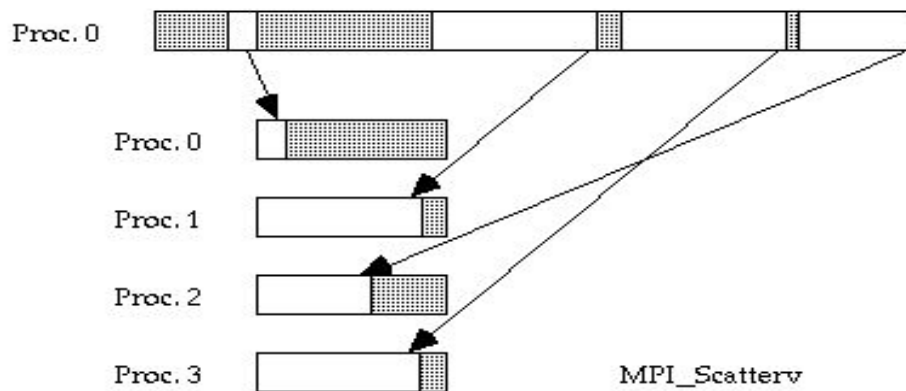
- Contiguous data
- Uniform message size



MPI Scatterv, MPI Gatherv

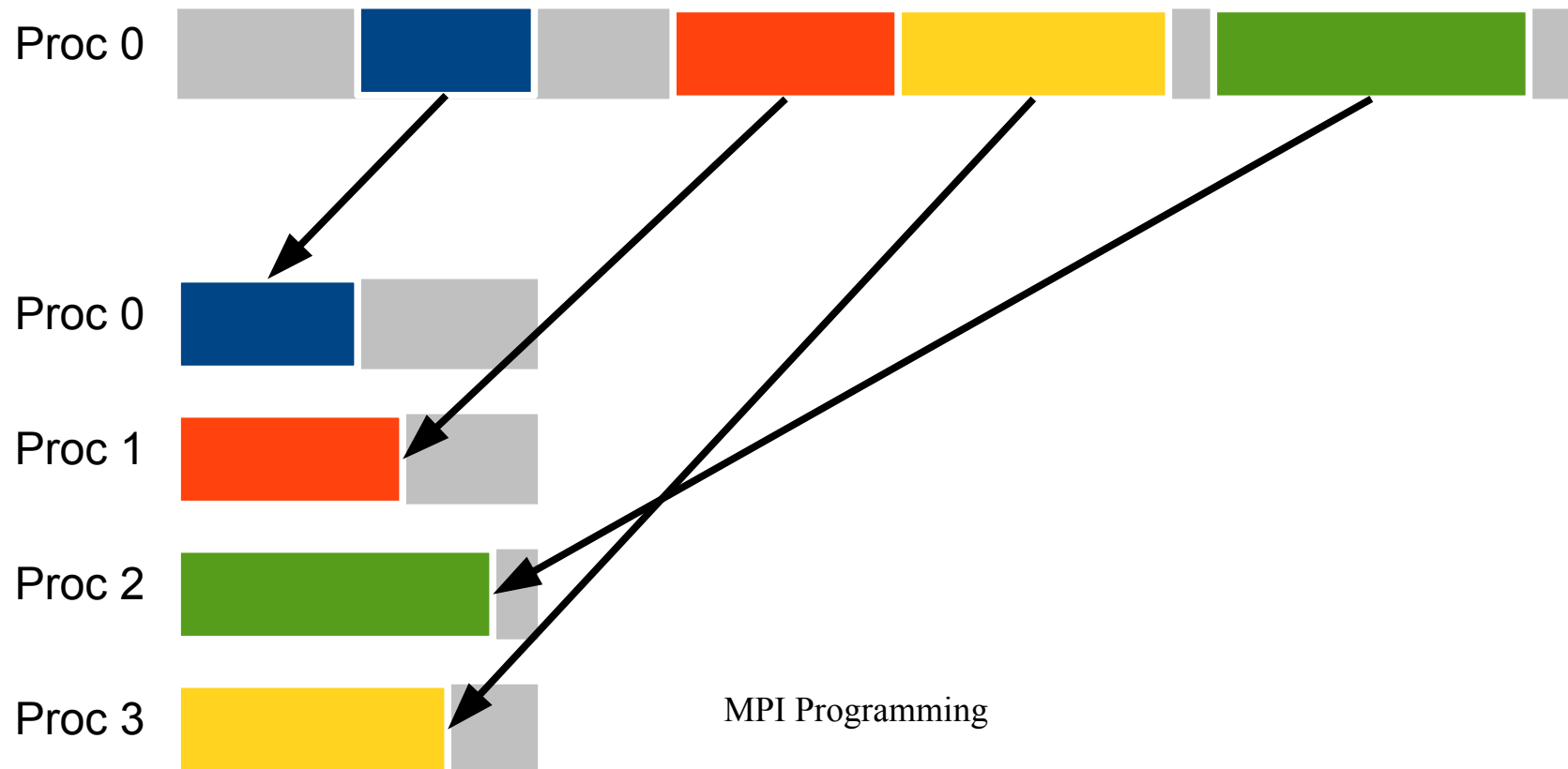
For many collective functions there are extended functionalities.

For example it's possible to define the length of arrays to be scattered or gathered with MPI_Scatterv and MPI_Gatherv.

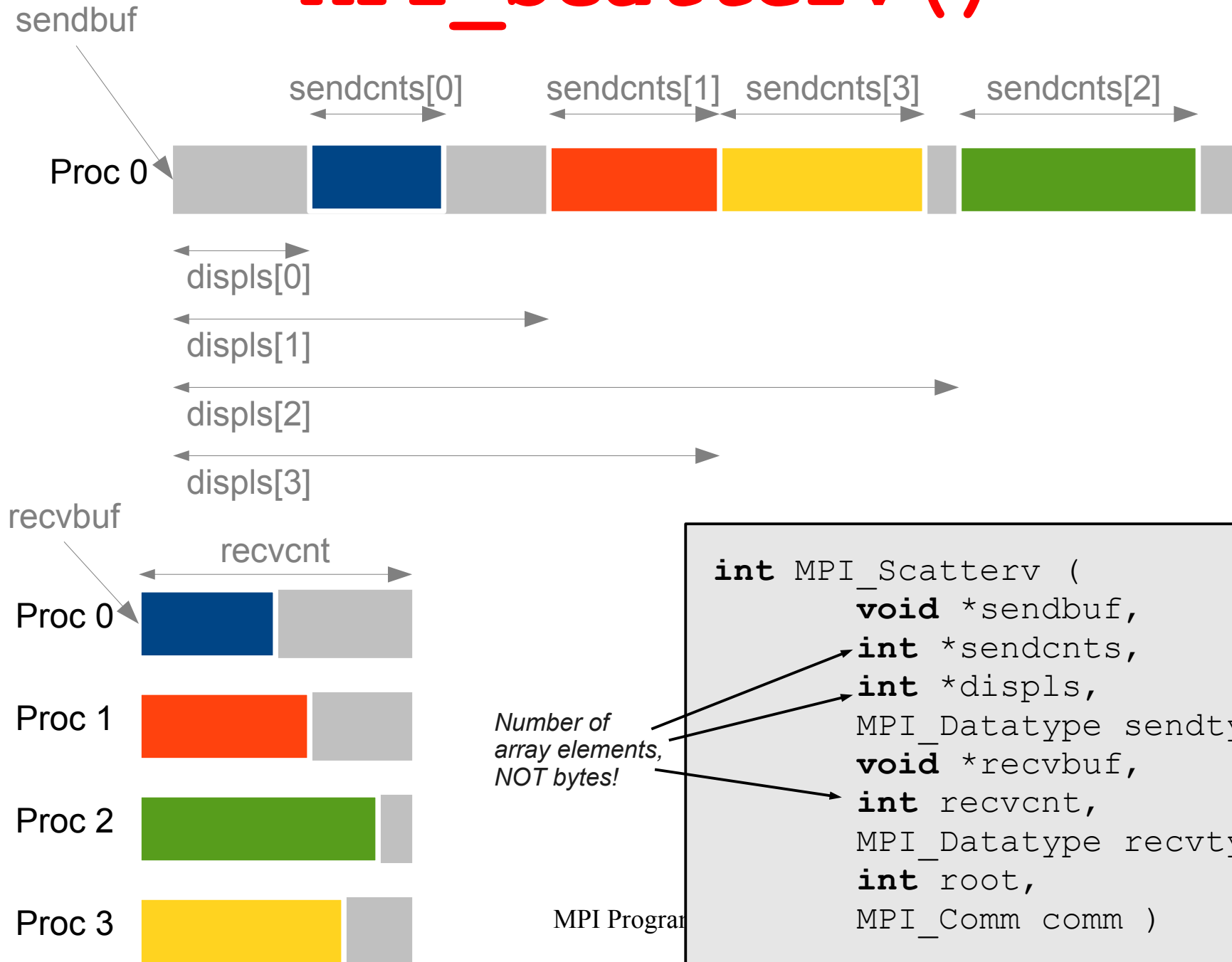


MPI_Scatterv() / MPI_Gatherv()

- Gaps are allowed between messages in source data
- Irregular message sizes are allowed
- Data can be distributed to processes in any order



MPI_Scatterv()



MPI Scaterv

C:

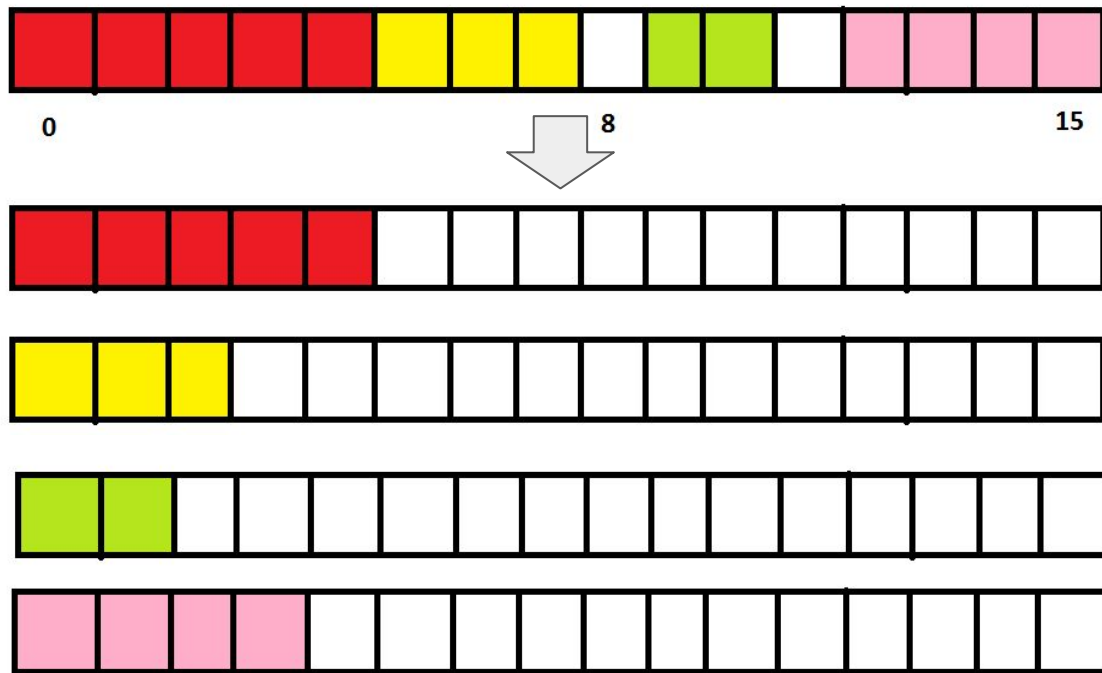
```
int MPI_Scatterv(const void *sendbuf, const int sendcounts[],  
                const int displs[], MPI_Datatype sendtype,  
                void *recvbuf, int recvcount, MPI_Datatype recvtype,  
                int root, MPI_Comm comm)
```

Fortran:

```
MPI_SCATTERV(SENDBUF, SENDCOUNTS, DISPLS, SENDTYPE,  
            RECVBUF, RECVCOUNT, RECVTYPE,  
            ROOT, COMM, IERROR)  
  
INTEGER    SENDCOUNTS(*), DISPLS(*), SENDTYPE  
INTEGER    RECVCOUNT, RECVTYPE, ROOT, COMM, IERROR
```

MPI Scaterv

sendcounts=(5,3,2,4) displs=(0,5,9,12)



Example

```
int sendbuf[] = {10, 11, 12, 13, 14, 15, 16}; /* at master */
int displs[] = {3, 0, 1}; /* assume P=3 MPI processes */
int sendcnts[] = {3, 1, 4};
int recvbuf[5];
...
MPI_Scatterv(sendbuf, sendcnts, displs, MPI_INT, recvbuf, 5,
MPI_INT, 0, MPI_COMM_WORLD);
```



Proc 0



Proc 1



Proc 2

recvbuf[]

MPI Gatherv

C:

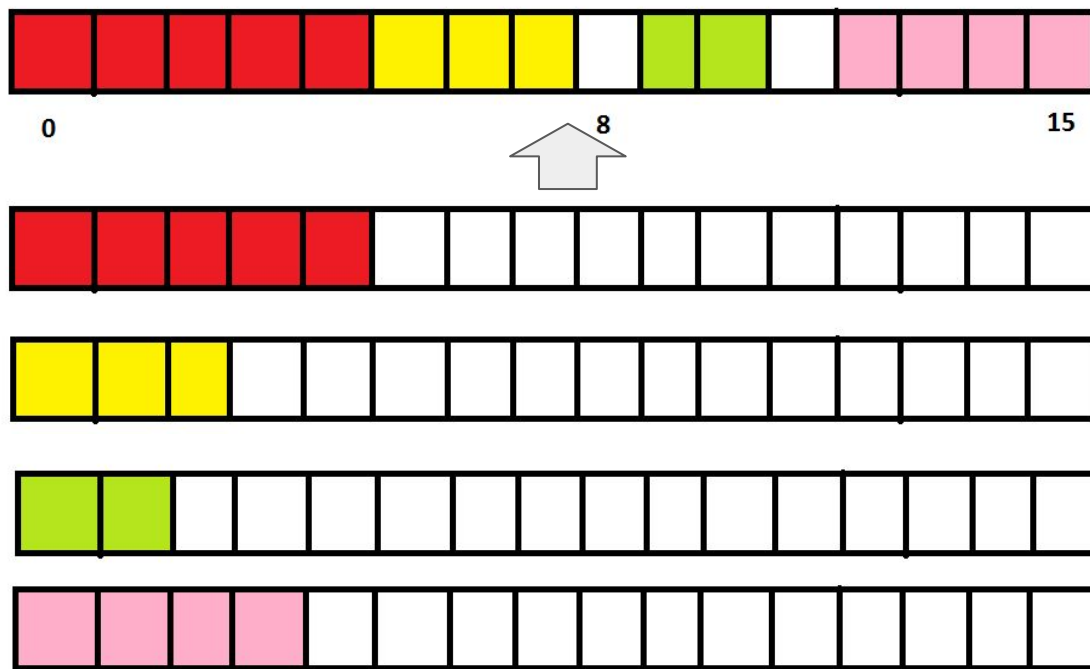
```
int MPI_Gatherv(const void *sendbuf, int sendcount,  
               MPI_Datatype sendtype, void *recvbuf, const int recvcounts[],  
               const int displs[], MPI_Datatype recvtype,  
               int root, MPI_Comm comm)
```

Fortran:

```
MPI_GATHERV(SENDBUF, SENDCOUNT, SENDTYPE,  
            RECVBUF, RECVCOUNTS, DISPLS, RECVTYPE,  
            ROOT, COMM, IERROR)  
  
INTEGER    SENDCOUNT, SENDTYPE, RECVCOUNTS(*), DISPLS(*)  
INTEGER    RECVTYPE, ROOT, COMM, IERROR
```

MPI Gatherv

recvcounts=(5,3,2,4) displs=(0,5,9,12)



MPI All to All

This function makes a redistribution of the content of each process in a way that each process know the buffer of all others. It is a way to implement the matrix data transposition.

```
int MPI_Alltoall(void *sendbuf, int sendcount, MPI_Datatype sendtype,
                void *recvbuf, int recvcount, MPI_Datatype recvtype,
                MPI_Comm comm)
```

a1	a2	a3	a4
----	----	----	----

b1	b2	b3	b4
----	----	----	----

c1	c2	c3	c4
----	----	----	----

d1	d2	d3	d4
----	----	----	----

a1	b1	c1	d1
----	----	----	----

a2	b2	c2	d2
----	----	----	----

a3	b3	c3	d3
----	----	----	----

a4	b4	c4	d4
----	----	----	----

MPI_Alltoall()

Each process performs a scatter operation

```
sendcnt = 1;  
recvcnt = 1;  
MPI_Alltoall(sendbuf, sendcnt, MPI_INT,  
              recvbuf, recvcnt, MPI_INT, MPI_COMM_WORLD);
```

	sendbuf[] (before)	recvbuf[] (after)								
Proc 0	<table><tr><td>1</td><td>2</td><td>3</td><td>4</td></tr></table>	1	2	3	4	<table><tr><td>1</td><td>5</td><td>9</td><td>13</td></tr></table>	1	5	9	13
1	2	3	4							
1	5	9	13							
Proc 1	<table><tr><td>5</td><td>6</td><td>7</td><td>8</td></tr></table>	5	6	7	8	<table><tr><td>2</td><td>6</td><td>10</td><td>14</td></tr></table>	2	6	10	14
5	6	7	8							
2	6	10	14							
Proc 2	<table><tr><td>9</td><td>10</td><td>11</td><td>12</td></tr></table>	9	10	11	12	<table><tr><td>3</td><td>7</td><td>11</td><td>15</td></tr></table>	3	7	11	15
9	10	11	12							
3	7	11	15							
Proc 3	<table><tr><td>13</td><td>14</td><td>15</td><td>16</td></tr></table>	13	14	15	16	<table><tr><td>4</td><td>8</td><td>12</td><td>16</td></tr></table>	4	8	12	16
13	14	15	16							
4	8	12	16							

Reduction

Reduction operations permits us to

- Collect data from each process
- Reduce the data to a single value
- Store the result on the root process (MPI_Reduce) or
- Store the result on all processes (MPI_Allreduce)

C:

```
int MPI_Reduce(const void *sendbuf,
               void *recvbuf, int count,
               MPI_Datatype datatype,
               MPI_Op op, int root,
               MPI_Comm comm);
```

Predefined reduction operations

MPI op	Function
MPI_MAX	Maximum
MPI_MIN	Minimum
MPI_SUM	Sum
MPI_PROD	Product
MPI_LAND	Logical AND
MPI_BAND	Bitwise AND
MPI_LOR	Logical OR
MPI_BOR	Bitwise OR
MPI_LXOR	Logical exclusive OR
MPI_BXOR	Bitwise exclusive OR
MPI_MAXLOC	Maximum and location
MPI_MINLOC	Minimum and location

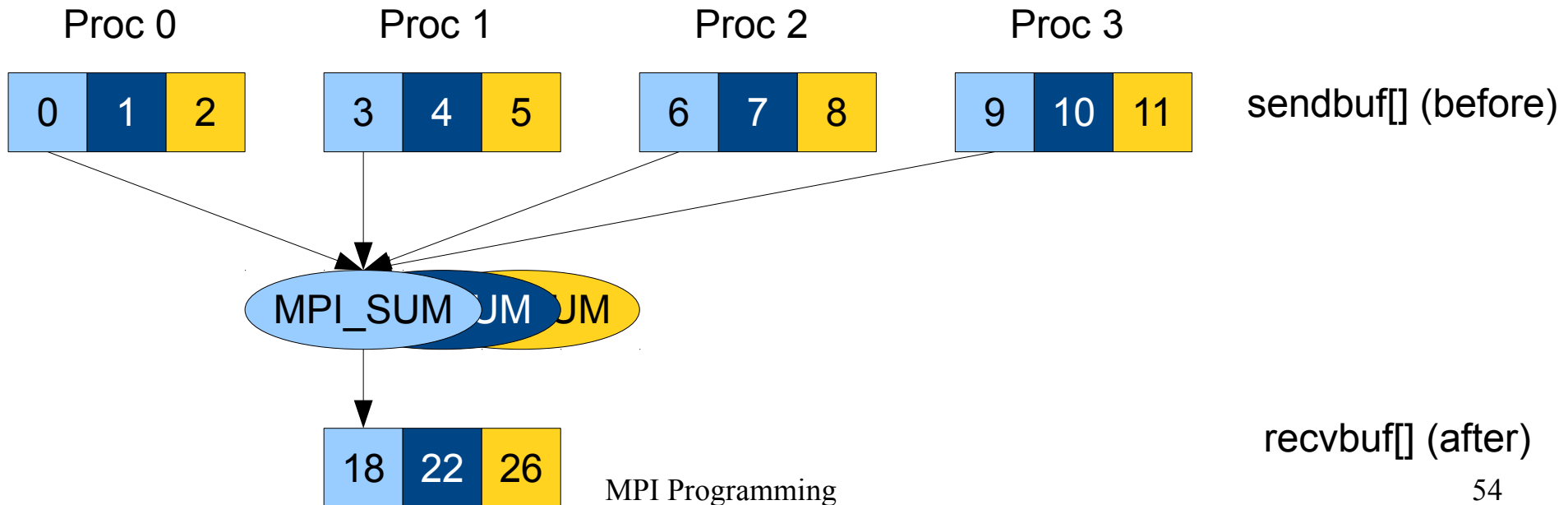
Example

```
PROGRAM reduce
  INCLUDE 'mpif.h'
  INTEGER ierr, myid, nproc, root
  REAL A(2), res(2)
  CALL MPI_INIT(ierr)
  CALL MPI_COMM_SIZE(MPI_COMM_WORLD, nproc, ierr)
  CALL MPI_COMM_RANK(MPI_COMM_WORLD, myid, ierr)
  root = 0
  A(1) = 2.0 * myid
  A(2) = 4.0 * myid
  CALL MPI_REDUCE(A, res, 2, MPI_REAL, MPI_SUM, root, MPI_COMM_WORLD, ierr)
  IF( myid .EQ. 0 ) THEN
    WRITE(6,*) myid, ': res(1)=', res(1), 'res(2)=', res(2)
  END IF
  CALL MPI_FINALIZE(ierr)
END
```

MPI_Reduce()

- If $\text{count} > 1$, `recvbuf[i]` is the reduction of all elements `sendbuf[i]` at the various processes

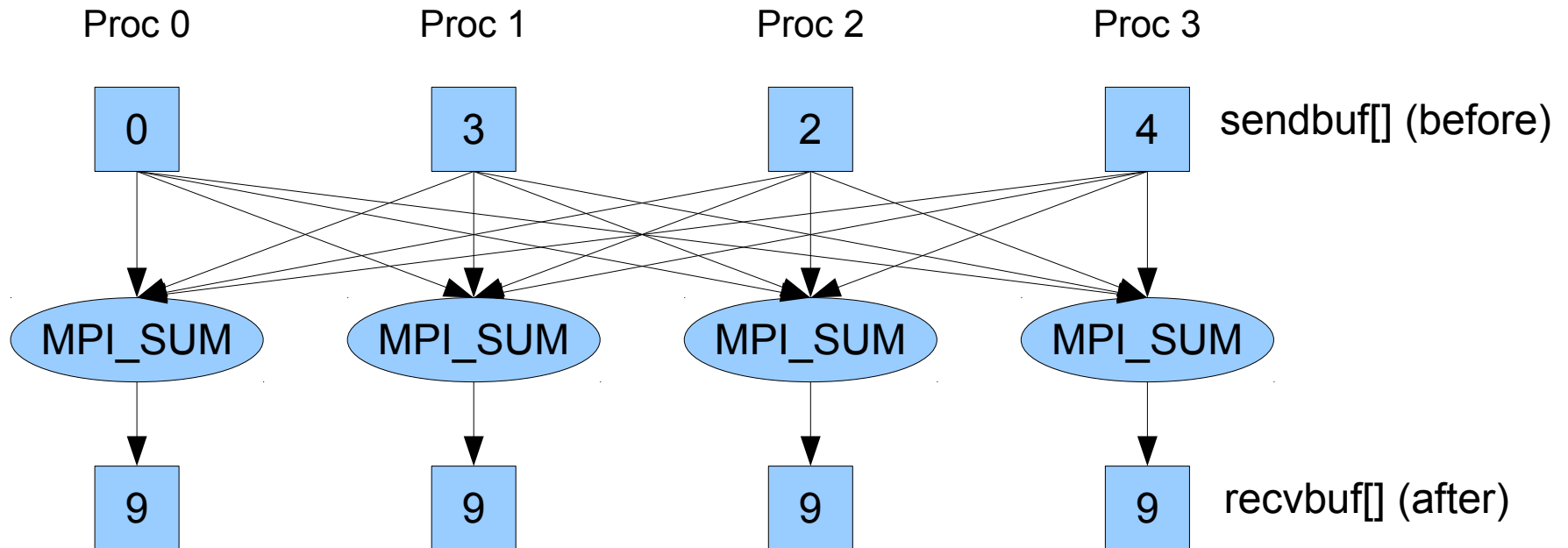
```
count = 3;  
dst = 1;  
MPI_Reduce(sendbuf, recvbuf, count, MPI_INT, MPI_SUM, dst, MPI_COMM_WORLD);
```



MPI_Allreduce()

Performs a reduction and place result in all processes

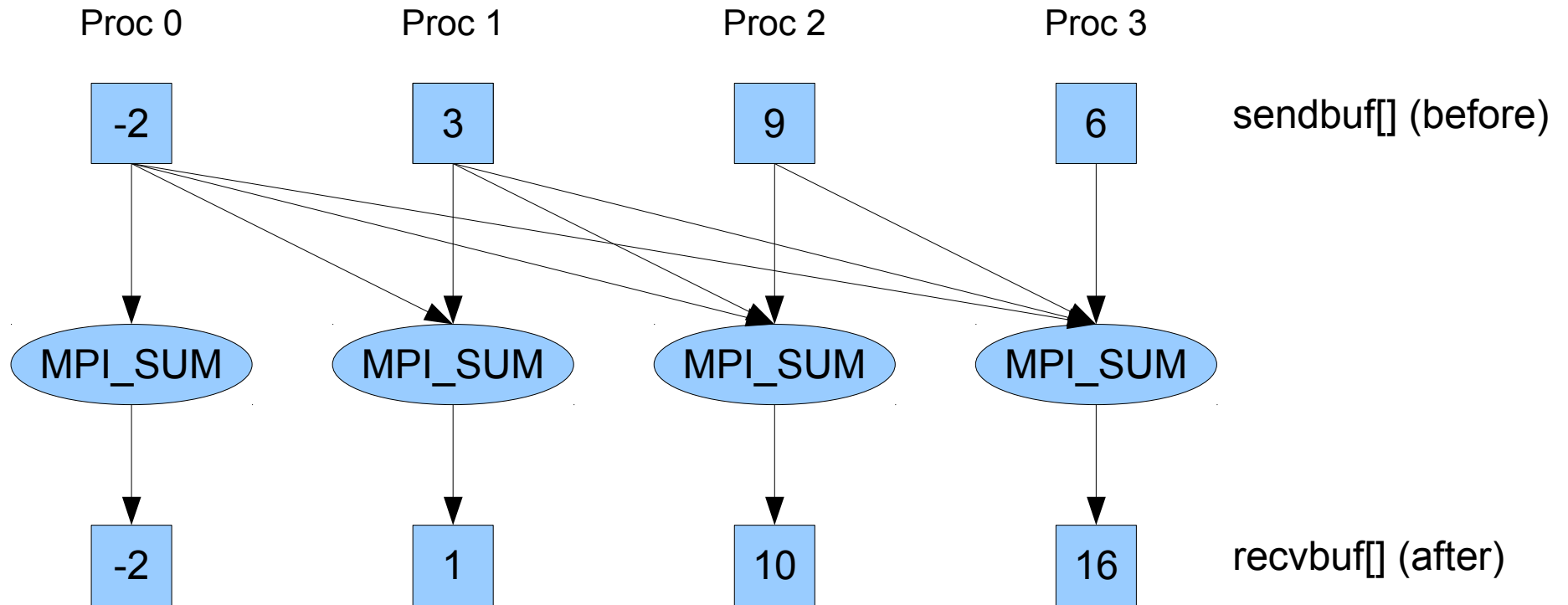
```
count = 1;  
MPI_Allreduce(sendbuf, recvbuf, count, MPI_INT, MPI_SUM, MPI_COMM_WORLD);
```



MPI_Scan()

Compute the inclusive scan

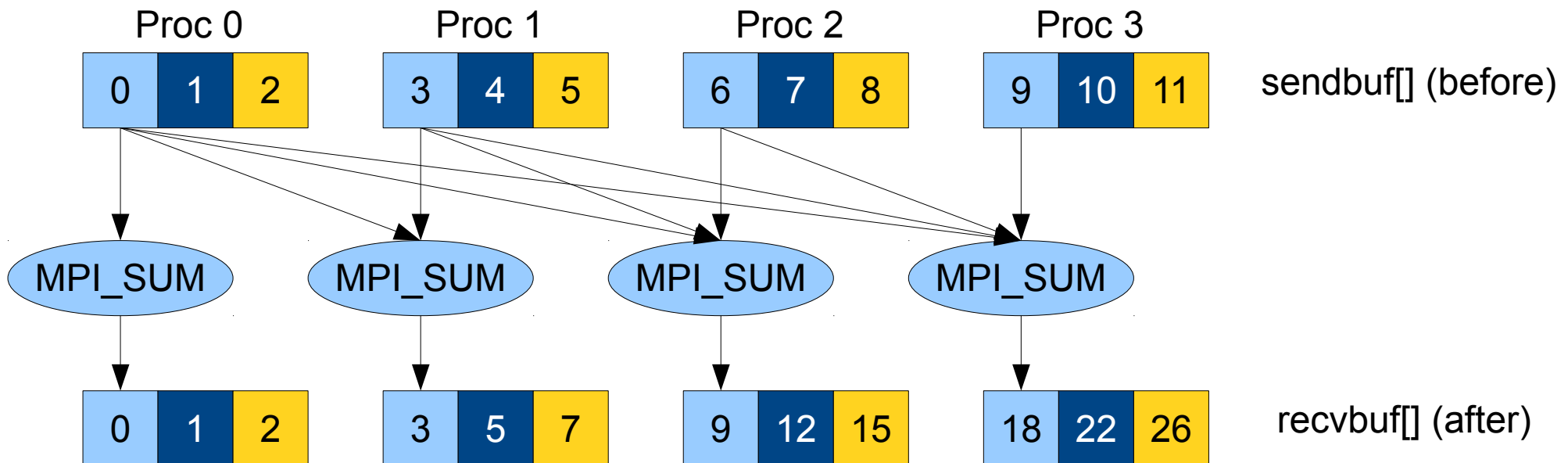
```
count = 1;  
MPI_Scan(sendbuf, recvbuf, count, MPI_INT, MPI_SUM, MPI_COMM_WORLD);
```



MPI_Scan()

- If $\text{count} > 1$, `recvbuf[i]` at proc. j is the scan of all elements `sendbuf[i]` at the first j processes (incl.)

```
count = 3;  
MPI_Scan(sendbuf, recvbuf, count, MPI_INT, MPI_SUM, MPI_COMM_WORLD);
```



Collective Communication Routines / 1

MPI_Barrier(comm)

- Synchronization operation. Creates a barrier synchronization in a group. Each process, when reaching the MPI_Barrier call, blocks until all processes in the group reach the same MPI_Barrier call. Then all processes can continue.

MPI_Bcast(&buffer, count, datatype, root, comm)

- Broadcasts (sends) a message from the process with rank "root" to all other processes in the group.

MPI_Scatter(&sendbuf, sendcnt, sendtype, &recvbuf, recvcnt, recvtype, root, comm)

- Distributes distinct messages from a single source process to each process in the group

MPI_Gather(&sendbuf, sendcnt, sendtype, &recvbuf, recvcount, recvtype, root, comm)

- Gathers distinct messages from each process in the group to a single destination process. This routine is the reverse operation of MPI_Scatter

MPI_Allgather(&sendbuf, sendcount, sendtype, &recvbuf, recvcount, recvtype, comm)

- Concatenation of data to all processes in a group. Each process in the group, in effect, performs a one-to-all broadcasting operation within the group

MPI_Reduce(&sendbuf, &recvbuf, count, datatype, op, root, comm)

- Applies a reduction operation on all processes in the group and places the result in one process

Collective Communication Routines / 2

`MPI_Reduce_scatter(&sendbuf, &recvbuf, recvcount, datatype, op, comm)`

- Collective computation operation + data movement. First does an element-wise reduction on a vector across all processes in the group. Next, the result vector is split into disjoint segments and distributed across the processes. This is equivalent to an `MPI_Reduce` followed by an `MPI_Scatter` operation.

`MPI_Scan(&sendbuf, &recvbuf, count, datatype, op, comm)`

- Performs a scan with respect to a reduction operation across a process group.

`MPI_Alltoall(&sendbuf, sendcount, sendtype, &recvbuf, recvcount, recvtype, comm)`

- Data movement operation. Each process in a group performs a scatter operation, sending a distinct message to all the processes in the group in order by index.

Performance issues

- Much hidden communication takes place with collective communication.
- Hardware vendors work hard to provide optimized collective calls but performances will vary according to implementation.
- Because of forced synchronization, collective communications may not always be the best solution.

Some studies show that around 80% transfer time is in collectives.

A Brief Word on MPI-2 and MPI-3

Intentionally, the MPI-1 specification did not address several "difficult" issues. For reasons of expediency, these issues were deferred to a second specification, called MPI-2 in 1998. Key areas of new functionality in MPI-2:

- **Dynamic Processes** - extensions that remove the static process model of MPI. Provides routines to create new processes after job startup.
- **One-Sided Communications** - provides routines for one directional communications. Include shared memory operations (put/get) and remote accumulate operations.
- **Extended Collective Operations** - allows for the application of collective operations to inter-communicators
- **External Interfaces** - defines routines that allow developers to layer on top of MPI, such as for debuggers and profilers.
- **Additional Language Bindings** - describes C++ bindings and discusses Fortran-90 issues.
- **Parallel I/O** - describes MPI support for parallel I/O.

A Brief Word on MPI-2 and MPI-3

The MPI-3 standard was adopted in 2012, and contains significant extensions to MPI-1 and MPI-2 functionality including:

- **Nonblocking Collective Operations** - permits tasks in a collective to perform operations without blocking, possibly offering performance improvements.
- **New One-sided Communication Operations** .
- **Neighborhood Collectives** - extends the distributed graph and Cartesian process topologies with additional communication power.
- **MPIT Tool Interface** - allows the MPI implementation to expose certain internal variables, counters, and other states to the user (most likely performance tools).
- **Matched Probe** - fixes an old bug in MPI-2 where one could not probe for messages in a multi-threaded environment.
- Presently efforts are devoted to a new standard, MPI-4
<https://www.mpi-forum.org/mpi-40/>

Some advanced topics

Alessandro Marani

a.marani@ Cineca .it

High Performance Computing department

Parallel programming with MPI

Derived Datatypes



Derived Data Types

What are they?

- Data types built from the basic MPI datatypes. Formally, the MPI Standard defines a general datatype as an object that specifies two things:
 - a sequence of basic datatypes
 - a sequence of integer (byte) displacements
- An easy way to represent such an object is as a sequence of pairs of basic datatypes and displacements. MPI calls this sequence a **typemap**.

```
typemap = { (type 0, displ 0), ... (type n-1, displ n-1) }
```

- But for most situations you do not need to worry about the **typemap**.
- Example: **typemap(mystruct) = { (char,0) , (int,8) , (double,16) }**

Derived Data Types

Why use them?

- Sometimes more convenient and efficient. For example, you may need to send messages that contain
 - non-contiguous data of a single type (e.g. a sub-block of a matrix)
 - contiguous data of mixed types (e.g., an integer count, followed by a sequence of real numbers)
 - non-contiguous data of mixed types.

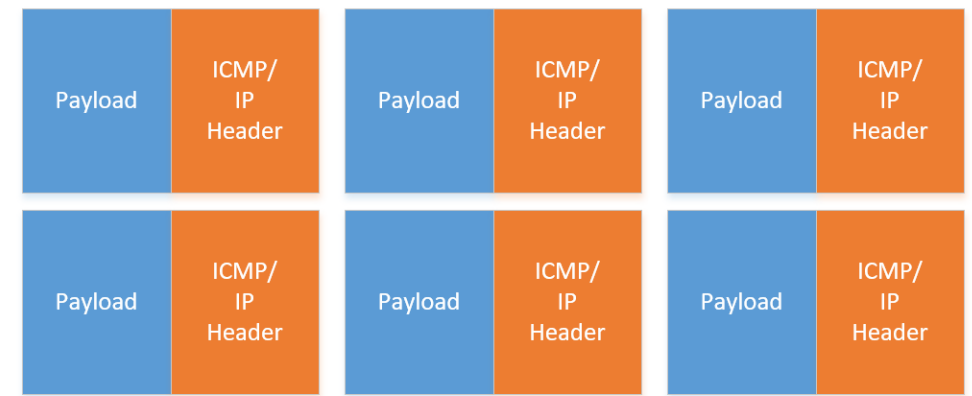
As well as improving program readability and portability they may improve performance.

Jumbo frames

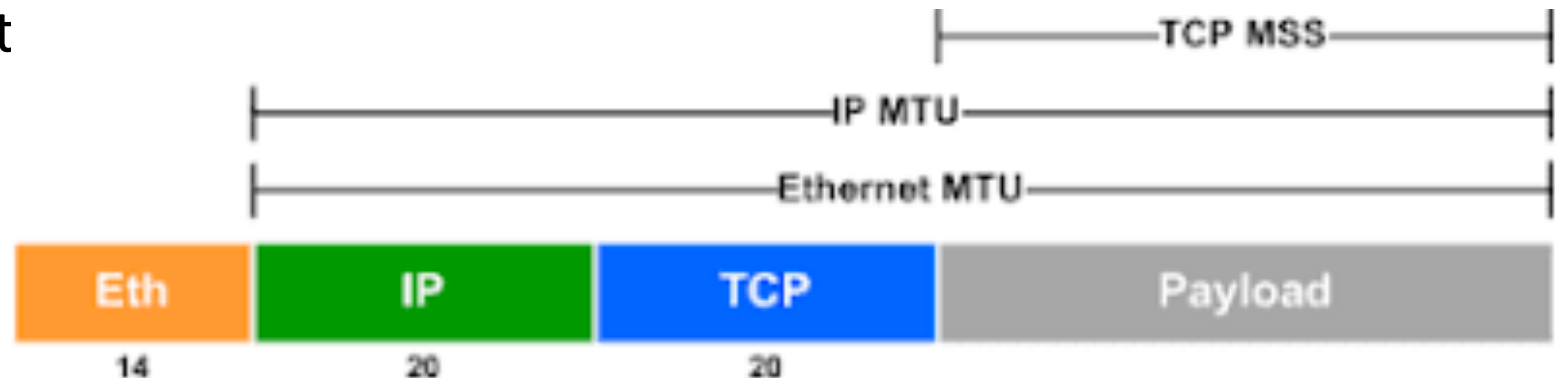
Sending a message costs $T(n) = \alpha + \beta n$

A Ethernet frame has a transmission unit (MTU) of 1526 byte, with a payload of up to 1472 byte. A jumbo frame, if supported, up to 9000, i.e. it can contain up to 6 messages.

If you merge several payload you can limit the overhead due to α , the latency, from $6 * \alpha$ to α . Easy to achieve for sending homogeneous arrays, not for heterogeneous dat

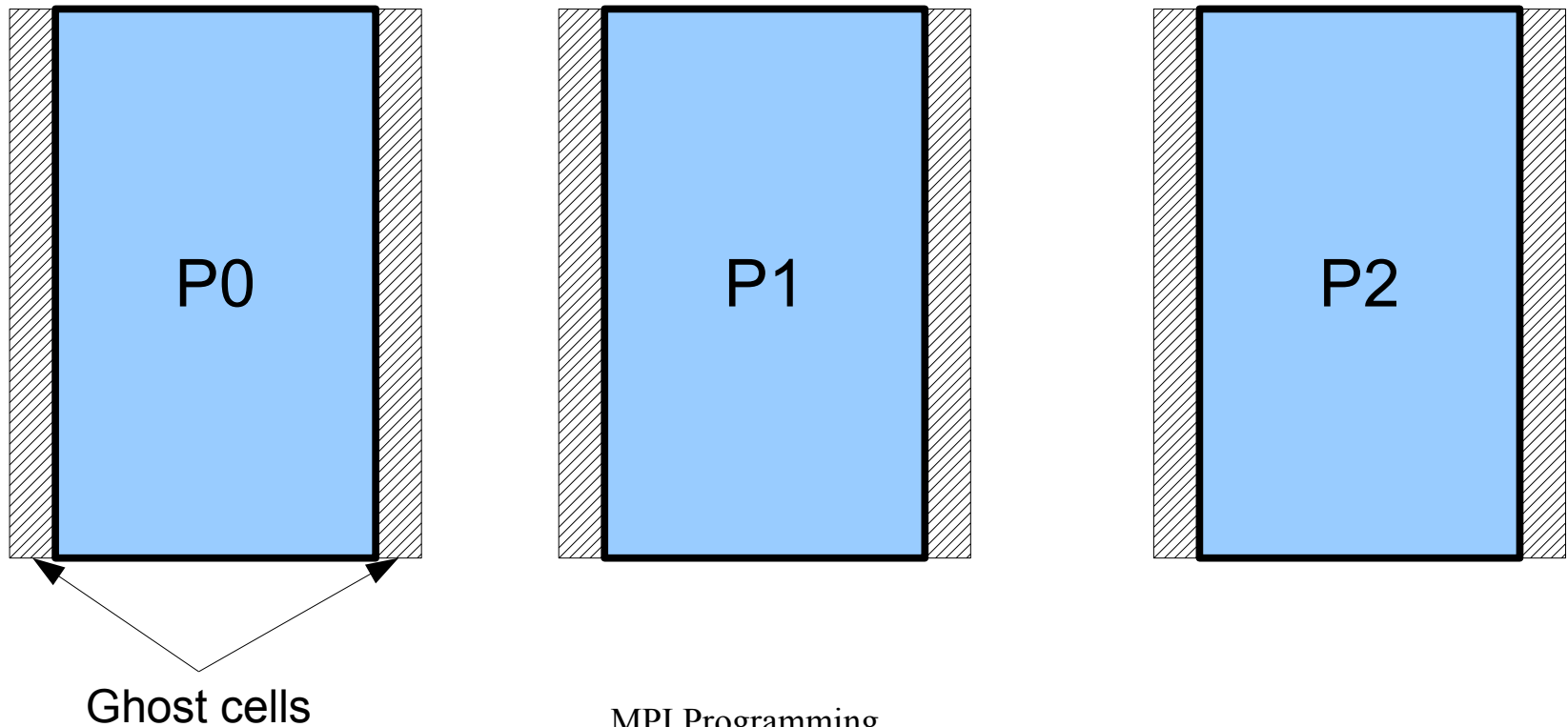


Or



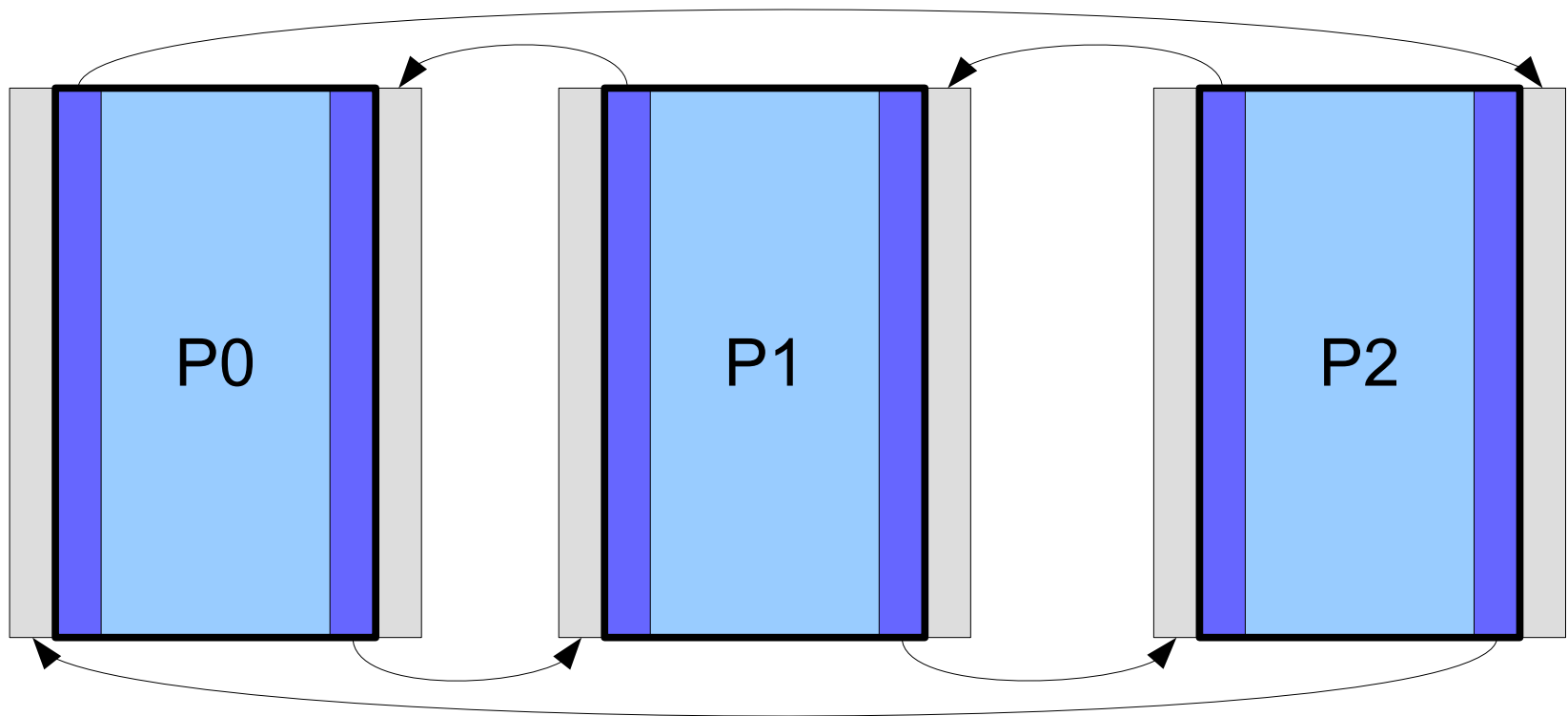
Example

- Let us consider a two-dimensional domain
- (*, Block) decomposition
 - with ghost cells along the vertical edges only



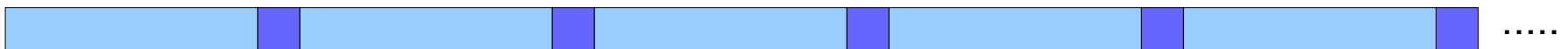
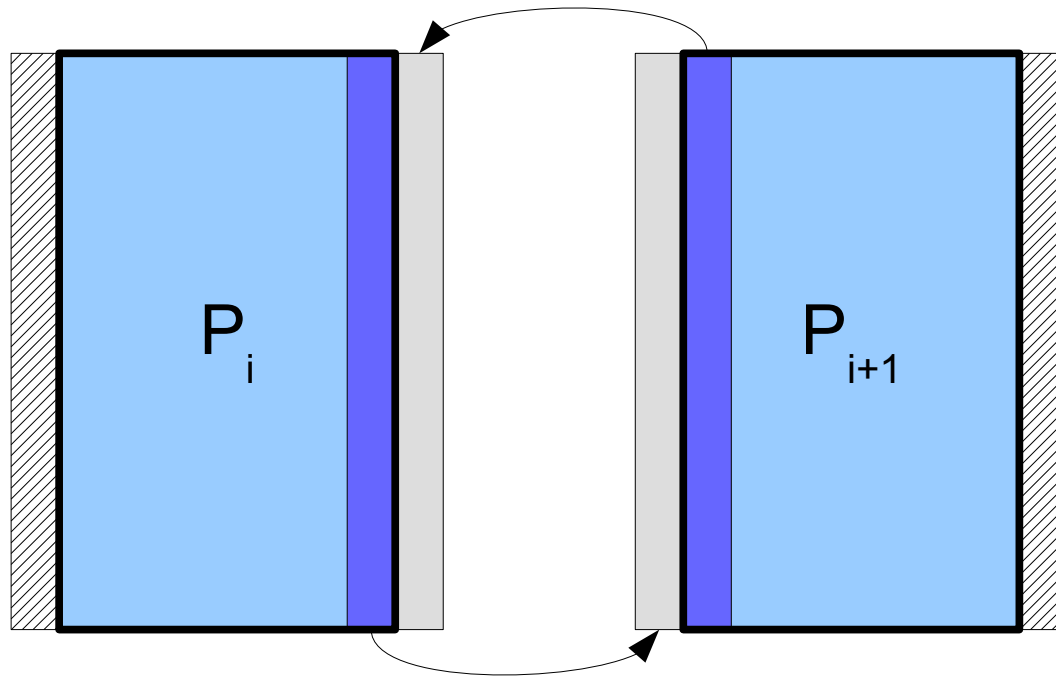
Example

- At each step, nodes must exchange their outer columns with neighbors



Example

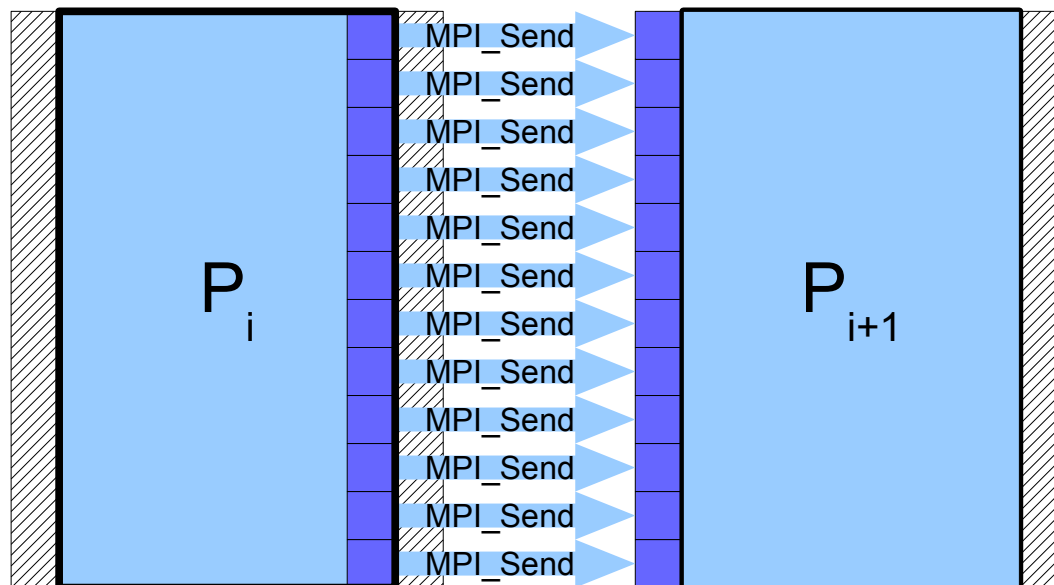
- In the C language, matrices are stored row-wise
 - Elements of the same column are not contiguous in memory





Example

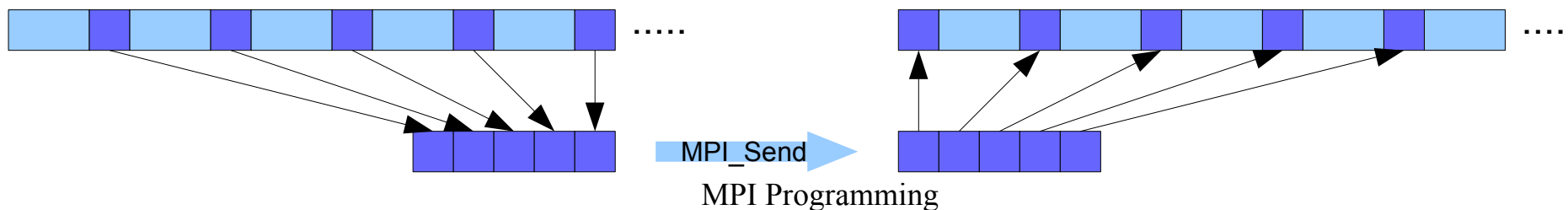
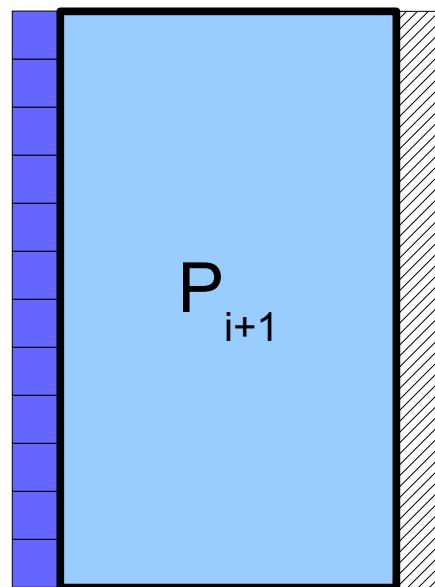
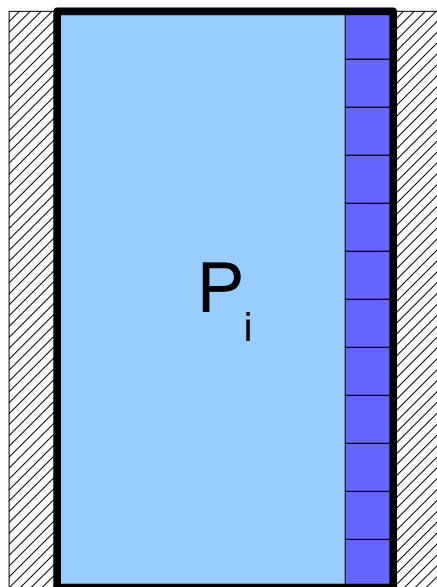
- **The BAD** solution: send each element with `MPI_Send` (or `MPI_Isend`)





Example

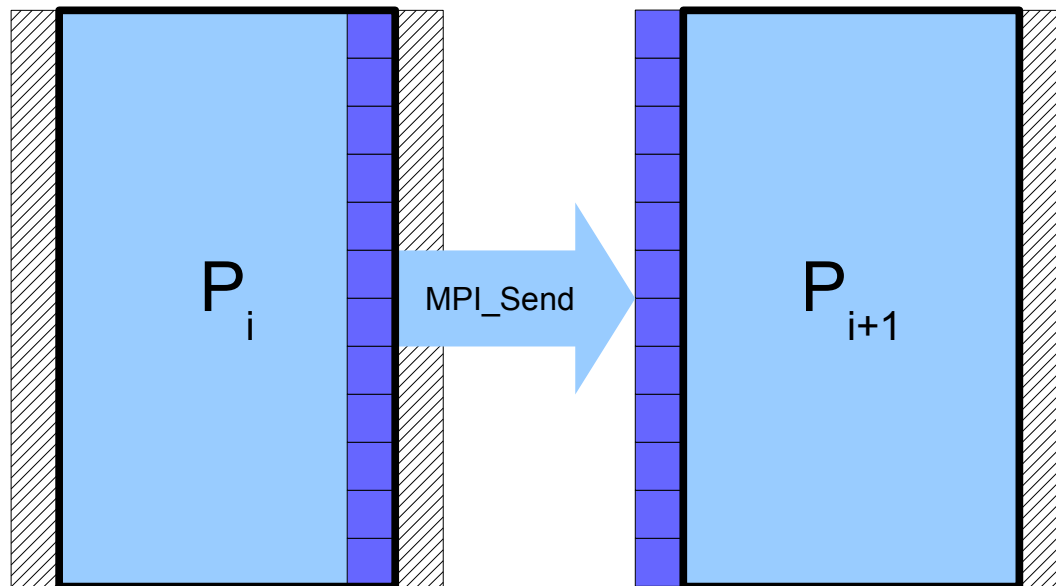
- **The UGLY** solution: copy the column into a temporary buffer; `MPI_Send()` the buffer; fill the destination column





Example

- **The GOOD** solution: define a new datatype for the column, and **MPI_Send** the column directly



How to use

1. Construct the datatype using a template or *constructor*.
2. Allocate the datatype.
3. Use the datatype.
4. Deallocate the datatype.

You must construct and allocate a datatype before using it. You are not required to use it or deallocate it, but it is recommended (there may be a limit).

Datatype constructors

- **MPI_Type_contiguous**
 - Simplest constructor. Makes count copies of an existing datatype
- **MPI_Type_vector, MPI_Type_hvector**
 - Like contiguous, but allows for regular gaps (stride) in the displacements. For MPI_Type_hvector the stride is specified in bytes.
- **MPI_Type_indexed, MPI_Type_hindexed**
 - An array of displacements of the input data type is provided as the map for the new data type. MPI_Type_hindexed is identical to MPI_Type_indexed except that offsets are specified in bytes
- **MPI_Type_struct**
 - The most general of all derived datatypes. The new data type is formed according to a completely defined map of the component data types

- C

- `int MPI_Type_commit (MPI_datatype *datatype)`
- `int MPI_Type_free (MPI_datatype *datatype)`

- FORTRAN

- `INTEGER DATATYPE, MPIERROR`
- `MPI_TYPE_COMMIT(DATATYPE, MPIERROR)`
- `MPI_TYPE_FREE(DATATYPE, MPIERROR)`

- C Example

- `MPI_Type_vector(count, blocklength, stride, oldtype, &newtype);`
- `MPI_Type_commit (&newtype);`
- `MPI_Send(buffer, 1, newtype, dest, tag, comm);`

MPI_TYPE_CONTIGUOUS

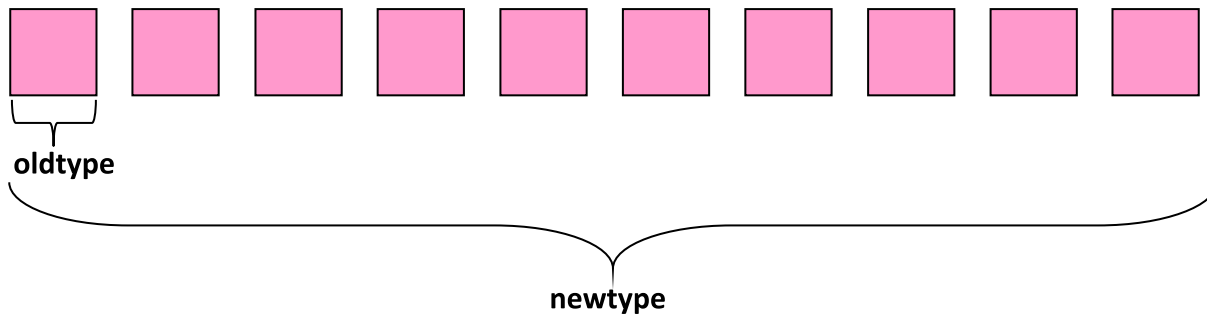
`MPI_TYPE_CONTIGUOUS (count, oldtype, newtype)`

IN count: replication count (non-negative integer)

IN oldtype: old datatype (handle)

OUT newtype: new datatype (handle)

- `MPI_TYPE_CONTIGUOUS` constructs a typemap consisting of the **replication** of a **datatype** into contiguous locations.
- `newtype` is the datatype obtained by concatenating count copies of `oldtype`.



MPI_TYPE_VECTOR

MPI_TYPE_VECTOR (count, blocklength, stride, oldtype, newtype)

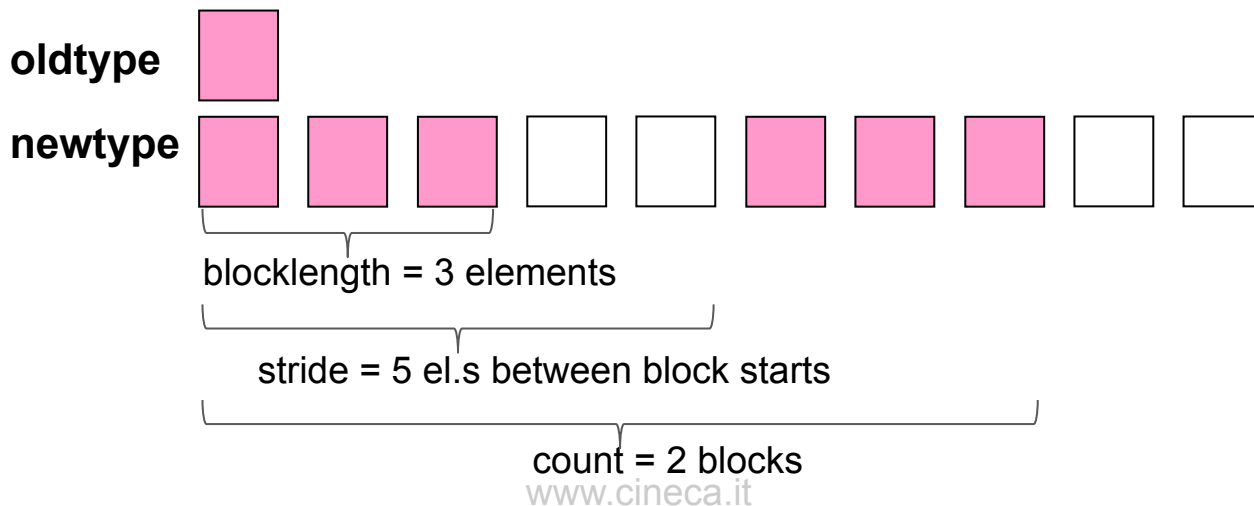
IN count: Number of blocks (non-negative integer)

IN blocklen: Number of elements in each block (non-negative integer)

IN stride: Number of elements (NOT bytes) between start of each block (integer)

IN oldtype: Old datatype (handle)

OUT newtype: New datatype (handle)



Example 1 – A rowtype

```
count = 4;
MPI_Type_contiguous(count, MPI_FLOAT, &rowtype);
```

1.0	2.0	3.0	4.0
5.0	6.0	7.0	8.0
9.0	10.0	11.0	12.0
13.0	14.0	15.0	16.0

$a[4][4]$

```
MPI_Send(&a[2][0], 1, rowtype, dest, tag, comm);
```

9.0	10.0	11.0	12.0
-----	------	------	------

1 element of
rowtype

Example 2 - columntype

```
count = 4; blocklength = 1; stride = 4;
MPI_Type_vector(count, blocklength, stride, MPI_FLOAT,
               &columntype);
```

1.0	2.0	3.0	4.0
5.0	6.0	7.0	8.0
9.0	10.0	11.0	12.0
13.0	14.0	15.0	16.0

$a[4][4]$

```
MPI_Send(&a[0][1], 1, columntype, dest, tag, comm);
```

2.0	6.0	10.0	14.0
-----	-----	------	------

1 element of
columntype

Quiz

```
int count = 4, blocklen = 2, stride = 4;  
MPI_Datatype newtype;  
MPI_Type_vector(count, blocklen, stride, MPI_FLOAT, &newtype);  
MPI_Type_commit(&newtype);
```

1.0	2.0	3.0	4.0
5.0	6.0	7.0	8.0
9.0	10.0	11.0	12.0
13.0	14.0	15.0	16.0

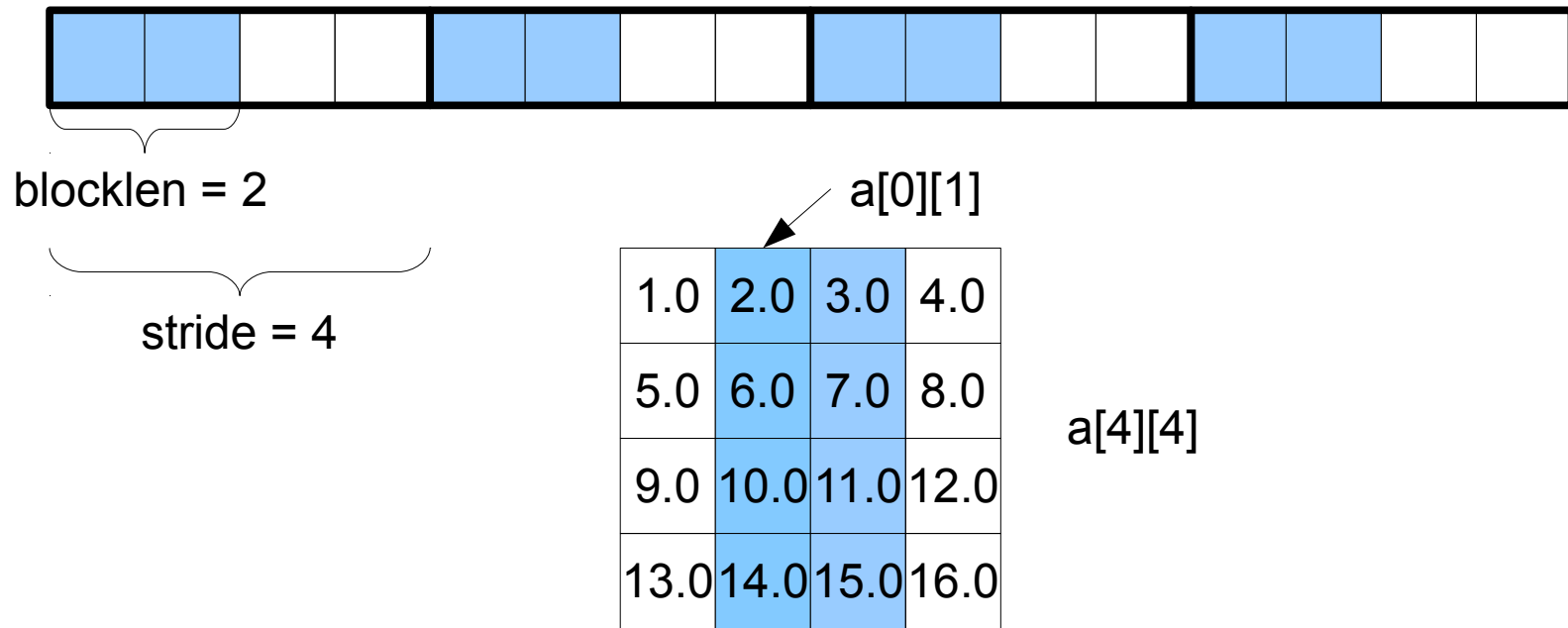
a[4][4]

```
MPI_Send(&a[0][1], 1, newtype, dest, tag, MPI_COMM_WORLD);
```

Which data are being transmitted?

Quiz

```
int count = 4, blocklen = 2, stride = 4;  
MPI_Datatype newtype;  
MPI_Type_vector(count, blocklen, stride, MPI_FLOAT, &newtype);  
MPI_Type_commit(&newtype);
```



```
MPI_Send(&a[0][1], 1, newtype, dest, tag, MPI_COMM_WORLD);
```

2.0	3.0	6.0	7.0	10.0	11.0	14.0	15.0
-----	-----	-----	-----	------	------	------	------

Quiz

```
int count = 3, blocklen = 1, stride = 5;  
MPI_Datatype newtype;  
MPI_Type_vector(count, blocklen, stride, MPI_FLOAT, &newtype);  
MPI_Type_commit(&newtype);
```

1.0	2.0	3.0	4.0
5.0	6.0	7.0	8.0
9.0	10.0	11.0	12.0
13.0	14.0	15.0	16.0

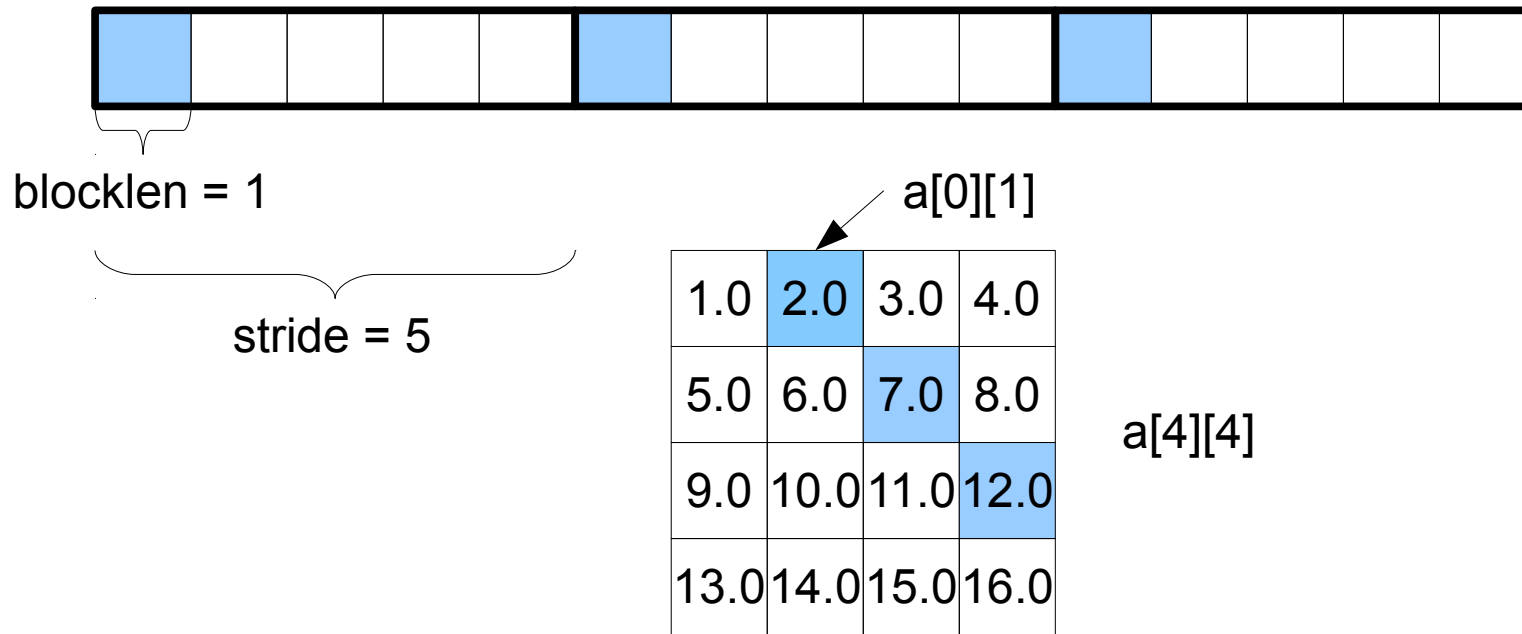
a[4][4]

```
MPI_Send(&a[0][1], 1, newtype, dest, tag, MPI_COMM_WORLD);
```

Which data are being transmitted?

Quiz

```
int count = 3, blocklen = 1, stride = 5;  
MPI_Datatype newtype;  
MPI_Type_vector(count, blocklen, stride, MPI_FLOAT, &newtype);  
MPI_Type_commit(&newtype);
```



```
MPI_Send(&a[0][1], 1, newtype, dest, tag, MPI_COMM_WORLD);
```

2.0	7.0	12.0
-----	-----	------

Quiz

```
int count = ???, blocklen = ???, stride = ???;  
MPI_Datatype newtype;  
MPI_Type_vector(count, blocklen, stride, MPI_FLOAT, &newtype);  
MPI_Type_commit(&newtype);
```

Fill the ??? with the parameters required to get the behavior below

1.0	2.0	3.0	4.0
5.0	6.0	7.0	8.0
9.0	10.0	11.0	12.0
13.0	14.0	15.0	16.0

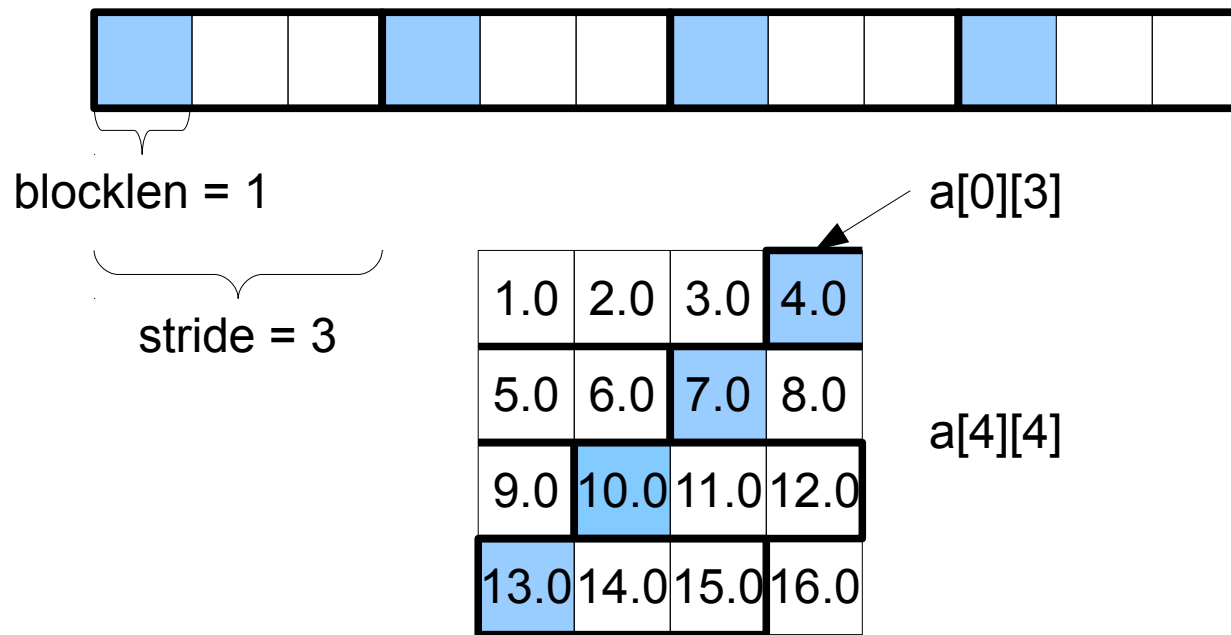
a[4][4]

```
MPI_Send(???????, 1, newtype, dest, tag, MPI_COMM_WORLD);
```

4.0	7.0	10.0	13.0
-----	-----	------	------

Quiz

```
int count = 4; blocklen = 1, stride = 3;  
MPI_Datatype newtype;  
MPI_Type_vector(count, blocklen, stride, MPI_FLOAT, &newtype);  
MPI_Type_commit(&newtype);
```



```
MPI_Send(&a[0][3], 1, newtype, dest, tag, MPI_COMM_WORLD);
```

4.0	7.0	10.0	13.0
-----	-----	------	------

MPI_GET_COUNT, MPI_GET_ELEMENTS

- Routines which return the number of "copies" of type datatype and the number of basic elements (often used after a MPI_RECV).

```
int MPI_Get_count( const MPI_Status *status, MPI_Datatype datatype, int *count )  
int MPI_Get_elements(const MPI_Status *status, MPI_Datatype datatype,  int *count)
```

MPI_TYPE_GET_EXTENT (Advanced)

- Returns the lower bound and extent of a datatype (i.e. upper bound + padding to align the datatype). Useful for creating new datatypes with MPI_TYPE_CREATE_RESIZED, for example.

MPI TYPE INDEXED

MPI_TYPE_INDEXED(count, array_of_blocklengths, array_of_displacements, oldtype, newtype)

IN count: number of blocks - also number of entries in array_of_blocklengths and array_of_displacements (non-negative integer)

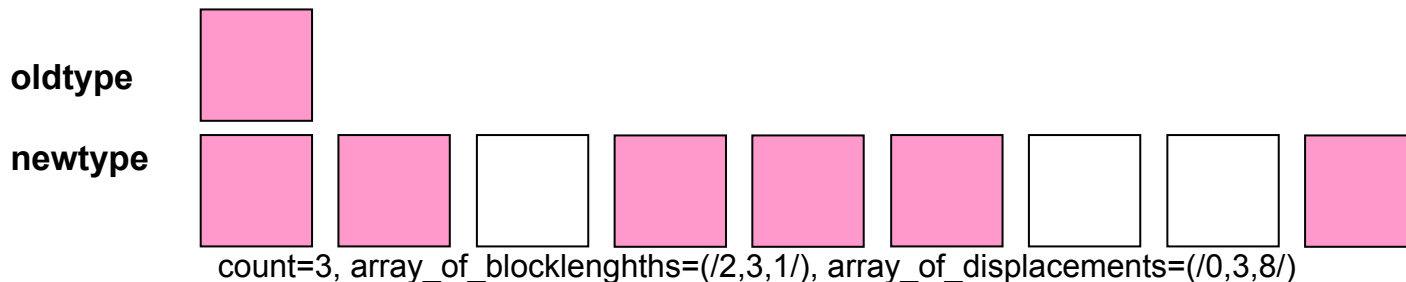
IN array_of_blocklengths: number of elements per block (array of non-negative integers)

IN array_of_displacements: displacement for each block, in multiples of oldtype extent (array of integer)

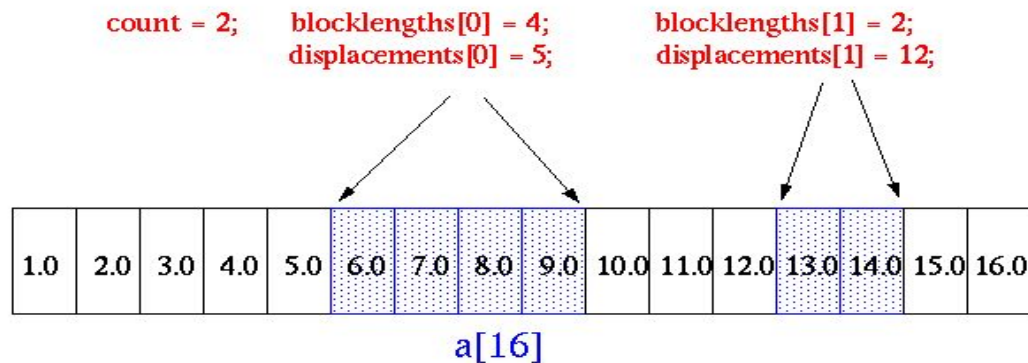
IN oldtype: old datatype (handle)

OUT newtype: new datatype (handle)

- Creates a new type from blocks comprising identical elements
- The size and displacements of the blocks can vary

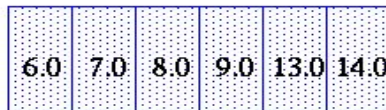


MPI TYPE INDEXED



```
MPI_Type_indexed(count, blocklengths, displacements, MPI_FLOAT, &indextype);
```

```
MPI_Send(&a, 1, indextype, dest, tag, comm);
```



1 element of
indextype

Combining custom datatypes

- The **oldtype** parameter of functions **MPI_Type_contiguous()**, **MPI_Type_vector()** and **MPI_Type_indexed()** can be another user-defined datatype

```
int count, blocklen, stride;
MPI_Datatype vec, vecvec;

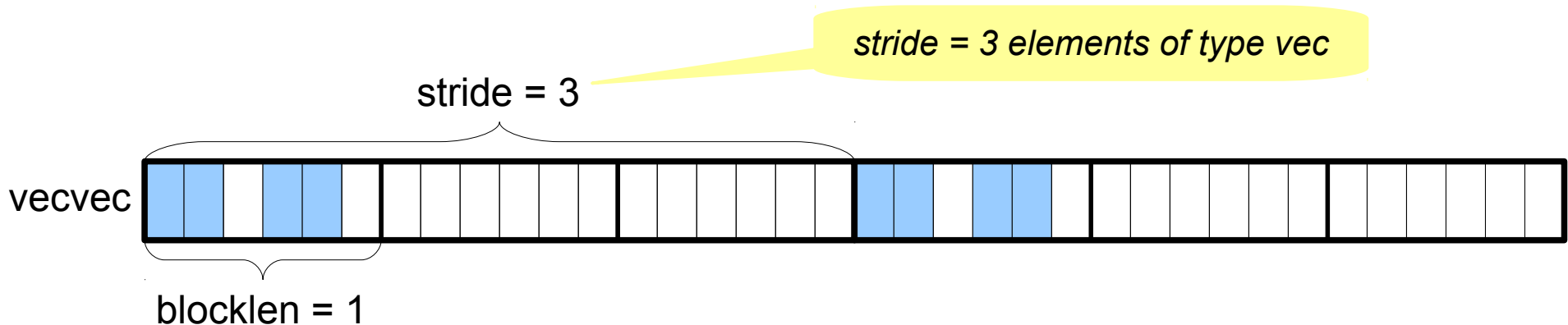
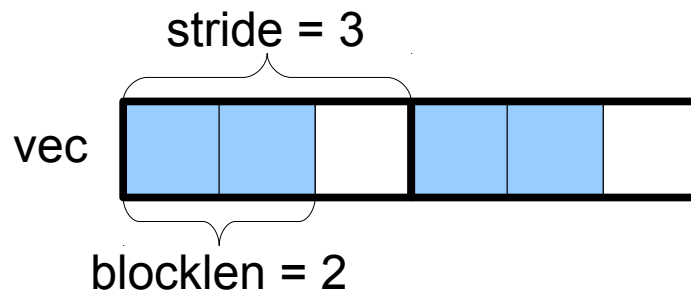
count = 2; blocklen = 2; stride = 3;
MPI_Type_vector( count, blocklen, stride, MPI_FLOAT, &vec);
MPI_Type_commit(&vec);
count = 2; blocklen = 1; stride = 3;
MPI_Type_vector( count, blocklen, stride, vec, &vecvec);
MPI_Type_commit(&vecvec);
```

```

int count, blocklen, stride;
MPI_Datatype vec, vecvec;

count = 2; blocklen = 2; stride = 3;
MPI_Type_vector( count, blocklen, stride, MPI_FLOAT, &vec);
MPI_Type_commit(&vec);
count = 2; blocklen = 1; stride = 3
MPI_Type_vector( count, blocklen, stride, vec, &vecvec);
MPI_Type_commit(&vecvec);

```



MPI TYPE SUBARRAY

```
MPI_TYPE_CREATE_SUBARRAY (ndims, array_of_sizes, array_of_subsizes, array_of_starts,  
                           order, oldtype, newtype)
```

IN ndims: number of array dimensions (positive integer)

IN array_of_sizes: number of elements of type oldtype in each
dimension of the full array (array of positive integers)

IN array_of_subsizes: number of elements of type oldtype in each
dimension of the subarray (array of positive integers)

IN array_of_starts: starting coordinates of the subarray in each
dimension (array of non-negative integers)

IN order: array storage order flag (state: MPI_ORDER_C or MPI_ORDER_FORTRAN)

IN oldtype: array element datatype (handle)

OUT newtype: new datatype (handle)

The subarray type constructor creates an MPI datatype describing an n dimensional subarray of an n -dimensional array. The subarray may be situated anywhere within the full array, and may be of any nonzero size up to the size of the larger array as long as it is confined within this array.

MPI TYPE SUBARRAY - EXAMPLE (C)

```
MPI_Type_create_subarray (ndims, array_of_sizes, array_of_subsizes,
                          array_of_starts, order, oldtype, newtype)
```

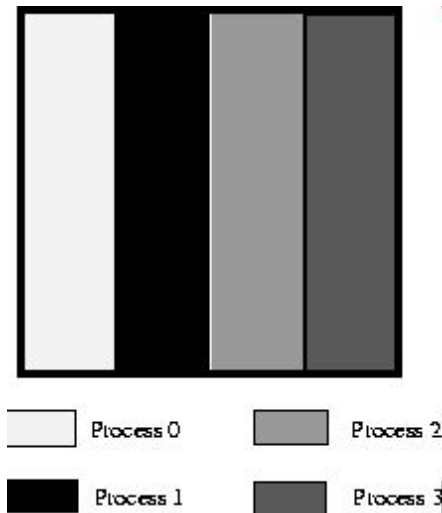
```
double subarray[100][25];
MPI_Datatype filetype;
int sizes[2], subsizes[2], starts[2];
int rank;

MPI_Comm_rank(MPI_COMM_WORLD, &rank);

sizes[0]=100; sizes[1]=100;
subsizes[0]=100; subsizes[1]=25;
starts[0]=0; starts[1]=rank*subsizes[1];
```

```
MPI_Type_create_subarray(2, sizes, subsizes, starts, MPI_ORDER_C, MPI_DOUBLE, &filetype);
```

```
MPI_Type_commit(&filetype);
```



SIZE AND EXTENT

The MPI datatype for structures – `MPI_TYPE_CREATE_STRUCT` – requires dealing with memory addresses and further concepts:

Typemap: pairs of basic types and displacements

Extent: The extent of a datatype is the span from the lower to the upper bound (including “holes”)

Size: The size of a datatype is the net number of bytes to be transferred (without “holes”)

int  char 

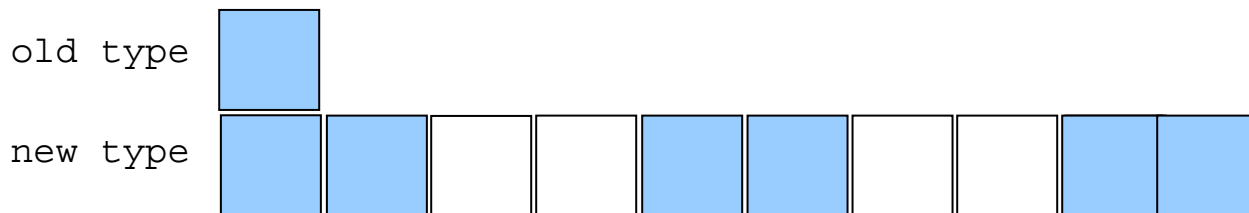
double 



derived datatype

SIZE AND EXTENT

- **Basic datatypes:**
 - size = extent = number of bytes used by the compiler
- **Derived datatypes:**
 - extent include holes but...
 - beware of the type vector: final holes are a figment of our imagination



- size = 6 x size of “old type”
- extent = 10 x extent of “old type”

QUERY SIZE AND EXTENT OF DATATYPE

- Returns the total number of bytes of the entry datatype

MPI_TYPE_SIZE (datatype, size)

IN datatype: datatype (handle)

OUT size: datatype size (integer)

- Returns the lower bound and the extent of the entry datatype

MPI_TYPE_EXTENT (datatype, extent)

IN datatype: datatype to get information on(handle)

OUT extent: extent of datatype (integer)

MPI TYPE STRUCT

```
MPI_TYPE_CREATE_STRUCT (count, array_of_blocklengths, array_of_displacements,  
                        array_of_oldtypes, newtype )
```

IN count: number of blocks (non-negative integer) -- also number of entries the following arrays

IN array_of_blocklengths: number of elements in each block
(array of non-negative integer)

IN array_of_displacements: byte displacement of each block (array of integer)

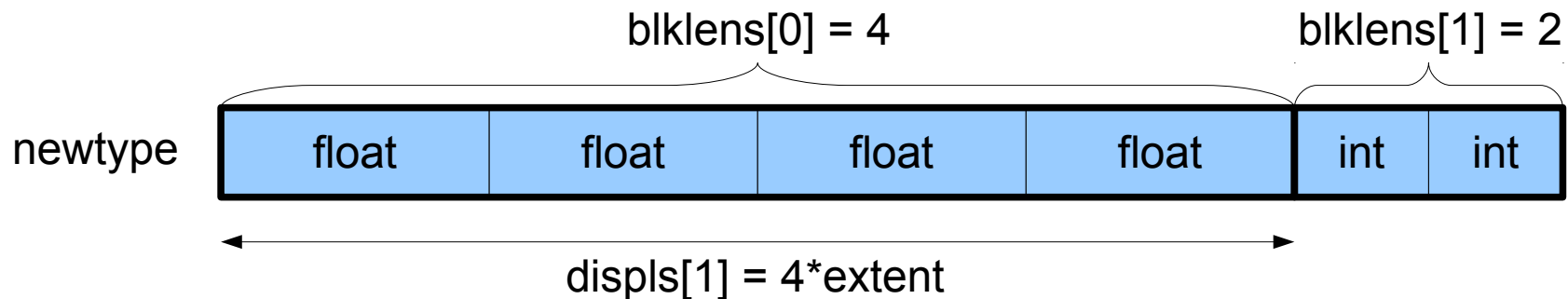
IN array_of_oldtypes: type of elements in each block
(array of handles to datatype objects)

OUT newtype: new datatype (handle)

- This subroutine returns a new datatype that represents count blocks. Each block is defined by an entry in array_of_blocklengths, array_of_displacements and array_of_types.
 - Displacements are expressed in bytes (since the type can change!)
 - To gather a mix of different datatypes scattered at many locations in space into one datatype that can be used for the communication.

MPI_Type_struct()

```
typedef struct {  
    float x, y, z, v;  
    int n, t;  
} particle_t;  
  
int count = 2; int blklen[] = {4, 2};  
MPI_Aint displs[2], lb, extent;  
MPI_Datatype oldtypes[2] = {MPI_FLOAT, MPI_INT}, newtype;  
MPI_Type_get_extent(MPI_FLOAT, &lb, &extent);  
displs[0] = 0; displs[1] = 4*extent;  
MPI_Type_struct(count, blklen, displs, oldtypes, &newtype);  
MPI_Type_commit(&newtype);
```



USING EXTENT (NOT SAFE)

C struct may be automatically padded by the compiler, e.g.

```
struct mystruct {  
    char a;  
    int b;  
    char c;  
} x
```



```
struct mystruct {  
    char a;  
    char gap_0[3];  
    int b;  
    char c;  
    char gap_1[3];  
} x
```

Using extents to handle structs is not safe! Get the addresses

MPI_GET_ADDRESS

MPI_GET_ADDRESS (location, address)

IN location: location in caller memory (choice)

OUT address: address of location (integer)

The address of the variable is returned, which can then be used to determine the correct relative displacements

Using this function helps with portability

USING DISPLACEMENTS

```
MPI_Datatype ParticleType;  
int count = 3;  
MPI_Datatype type[3] = {MPI_CHAR, MPI_DOUBLE, MPI_INT};  
int blocklen[3] = {1, 6, 7};  
MPI_Aint disp[3];
```

```
MPI_Get_address(&particle[0].class, &disp[0]);  
MPI_Get_address(&particle[0].d, &disp[1]);  
MPI_Get_address(&particle[0].b, &disp[2]);  
/* Make displacements relative */  
disp[2] -= disp[0]; disp[1] -= disp[0]; disp[0] = 0;
```

```
MPI_Type_create_struct (count, blocklen, disp, type, &ParticleType);  
MPI_Type_commit (&ParticleType);
```

```
MPI_Send(particle, 100, ParticleType, dest, tag, comm);  
MPI_Type_free (&ParticleType);
```

```
struct PartStruct {  
    char class;  
    double d[6];  
    int b[7];  
} particle[100];
```

Derived Datatype Summary

- Provide a portable and elegant way of communicating non-contiguous or mixed types in a message.
- By optimising how data is stored, should improve efficiency during MPI send and receive (perhaps avoiding buffering).
- Derived datatypes are built from basic MPI datatypes, according to a template. Can be used for many variables of the same form.
- Remember to commit the datatypes before using them.

Alessandro Marani

a.marani@cineca.it

High Performance Computing department

Parallel programming with MPI

Communicators and Groups



MPI communicators and groups

Many users are familiar with the mostly used communicator:

`MPI_COMM_WORLD`

A **communicator** can be thought as a handle to a **group**.

- a group is a ordered set of processes
 - each process is associated with a rank
 - ranks are contiguous and start from zero

Groups allow collective operations to be operated on a subset of processes

The group routines are primarily used to specify which processes should be used to construct a communicator.

MPI communicators and groups

Intracommunicators

are used for communications within a single group

Intercommunicators

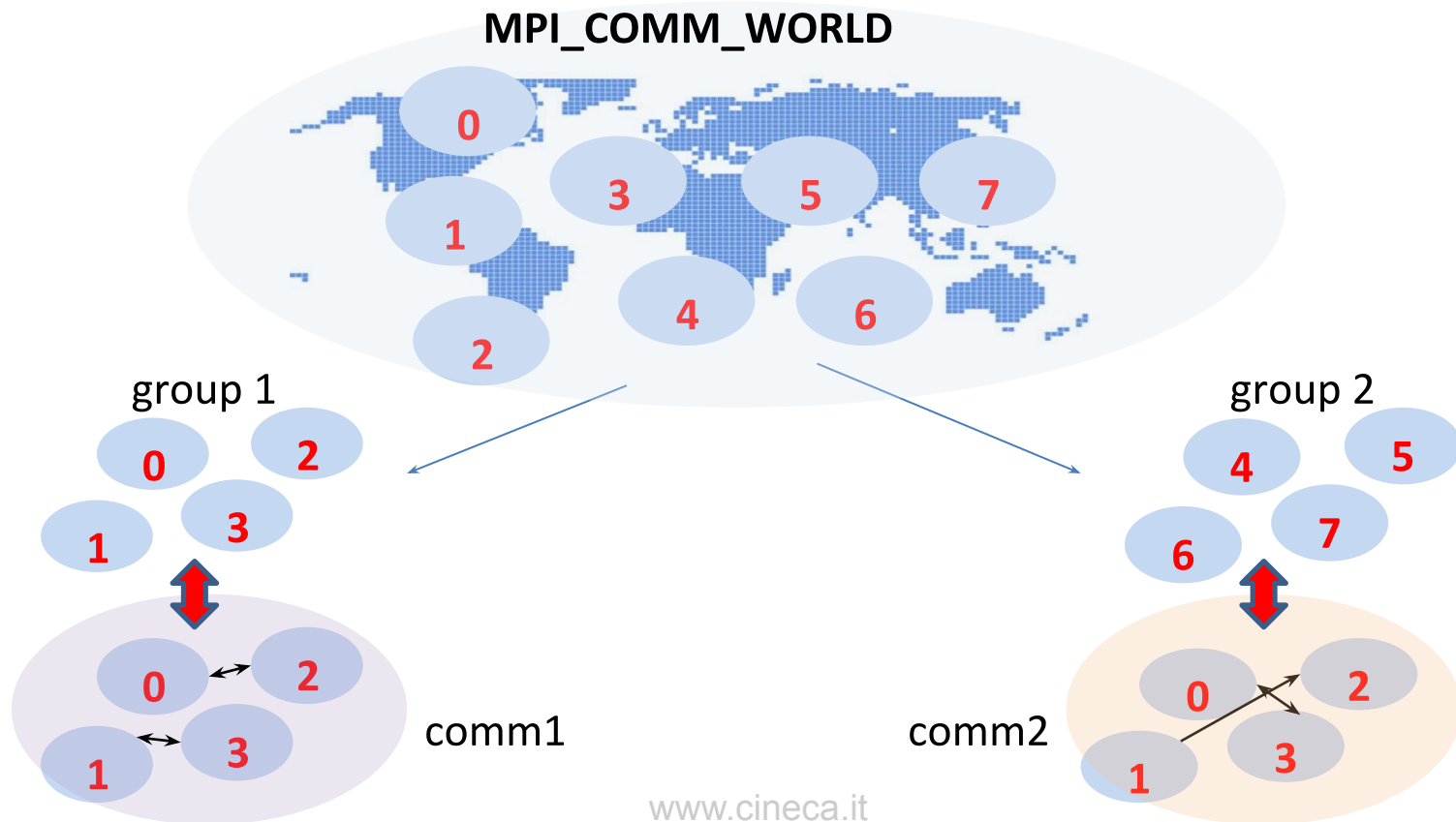
are used for communications between two disjoint groups

Using MPI Groups

Typical usage:

1. Extract handle of global group from `MPI_COMM_WORLD` using `MPI_Comm_group`
2. Form new group as a subset of global group using `MPI_Group_incl`
3. Create new communicator for new group using `MPI_Comm_create`
4. Determine new rank in new communicator using `MPI_Comm_rank`
5. Conduct communications using any MPI message passing routine
6. When finished, free up new communicator and group (optional) using `MPI_Comm_free` and `MPI_Group_free`

Group Creation



Group accessors

MPI_GROUP_SIZE

This routine returns the number of processes in the group

MPI_GROUP_RANK

This routine returns the rank of the calling process inside a given group

Group accessors

MPI_COMM_GROUP(comm,group,ierr)

This routine returns the group associated with the communicator comm

MPI_GROUP_UNION(group_a, group_b, newgroup, ierr)

This returns the ensemble union of group_a and group_b

MPI_GROUP_INTERSECTION(group_a, group_b, newgroup, ierr)

This returns the ensemble intersection of group_a and group_b

MPI_GROUP_DIFFERENCE(group_a, group_b, newgroup, ierr)

This returns in newgroup all processes in group_a that are not in group_b, ordered as in group_a

Group accessors

MPI_GROUP_INCL(group, n, ranks, newgroup, ierr)

This routine creates a new group that consists of all the n processes with ranks ranks[0]... ranks[n-1]

Example:

group = {a,b,c,d,e,f,g,h,i,j}

n = 5

ranks = {0,3,8,6,2}

newgroup = {a,d,i,g,c}

Group accessors

MPI_GROUP_EXCL(group,n,ranks,newgroup,ierr)

This routine returns a newgroup that consists of all the processes in the group after removing processes with ranks: ranks[0]..ranks[n-1]

Example:

group = {a,b,c,d,e,f,g,h,i,j}

n = 5

ranks = {0,3,8,6,2}

newgroup = {b,e,f,h,j}

Communicator accessors

MPI_COMM_SIZE(comm,size,ierr)

Returns the number of processes in the group associated with the comm

MPI_COMM_RANK(comm,rank,ierr)

Returns the rank of the calling process within the group associated with the comm

MPI_COMM_COMPARE(comm1,comm2,result,ierr)

Returns:

- MPI_IDENT if comm1 and comm2 are the same handle
- MPI_CONGRUENT if comm1 and comm2 have the same group attribute
- MPI_SIMILAR if the groups associated with comm1 and comm2 have the same members but in different rank order
- MPI_UNEQUAL otherwise

Communicator constructors

MPI_COMM_DUP(comm, newcomm, ierr)

This returns a communicator newcomm identical to the communicator comm

MPI_COMM_CREATE(comm, group, newcomm, ierr)

This collective routine must be called by all the process involved in the group associated with comm. It returns a new communicator that is associated with the group. MPI_COMM_NULL is returned to processes not in the group.

Note that group must be a subset of the group associated with comm!

Communicator constructors

MPI_COMM_SPLIT(comm, color, key, newcomm, ierr)

This routine creates as many new groups and communicators as there are distinct values of color.

(processes in the same color are in the same communicator)

The **rankings** in the new groups are determined by the value of the key.

MPI_UNDEFINED is used as the color for processes to not be included in any of the new groups

Example

Rank	0	1	2	3	4	5	6	7	8	9	10
Process	a	b	c	d	e	f	g	h	i	j	k
Color	U	3	1	1	3	7	3	3	1	U	3
Key	0	1	2	3	1	9	3	8	1	0	0

Both process a and j are returned MPI_COMM_NULL

3 new groups are created

{i, c, d}

{k, b, e, g, h}

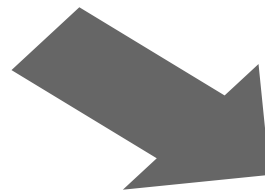
{f}

MPI_comm_split

MPI provides functions to manage and to create groups and communicators.

MPI_comm_split, for example, creates a communicator...

```
if (myid%2==0) {  
    color=1;  
} else {  
    color=2;  
}  
MPI_COMM_SPLIT(MPI_COMM_WORLD, color, myid,  
               &subcomm);  
MPI_COMM_RANK(subcomm, mynewid);  
printf("rank in MPICOMM_WORLD %d", myid,  
       "rank in Subcomm %d", mynewid);
```



I am rank 2 in MPI_COMM_WORLD, but 1 in Comm 1.
I am rank 7 in MPI_COMM_WORLD, but 3 in Comm 2.
I am rank 0 in MPI_COMM_WORLD, but 0 in Comm 1.
I am rank 4 in MPI_COMM_WORLD, but 2 in Comm 1.
I am rank 6 in MPI_COMM_WORLD, but 3 in Comm 1.
I am rank 3 in MPI_COMM_WORLD, but 1 in Comm 2.
I am rank 5 in MPI_COMM_WORLD, but 2 in Comm 2.
I am rank 1 in MPI_COMM_WORLD, but 0 in Comm 2.

Example

```
int MPI_Comm_split(MPI_Comm old_comm, int color, int key, MPI_Comm *new_comm)
```

For a 2D logical grid, create subgrids
of rows and columns

```
!! logical row number
```

```
irow = Iam/mcol
```

```
!! logical column number
```

```
jcol = mod(Iam, mcol)
```

```
comm2D = MPI_COMM_WORLD
```

```
call MPI_Comm_split(comm2D, irow, jcol, row_comm, ierr)
```

```
call MPI_Comm_split(comm2D, jcol, irow, col_comm, ierr)
```

Figure a.
2D logical Grid

(0)	(1)
(2)	(3)
(4)	(5)

Figure b.
3 Row Subgrids

(0) (0)	(1) (1)
(2) (0)	(3) (1)
(4) (0)	(5) (1)

Figure c.
2 Column Subgrids

(0) (0)	(1) (0)
(2) (1)	(3) (1)
(4) (2)	(5) (2)

Iam	0	1	2	3	4	5
irow	0	0	1	1	2	2
jcol	0	1	0	1	0	1



Virtual Topology

Virtual topology:

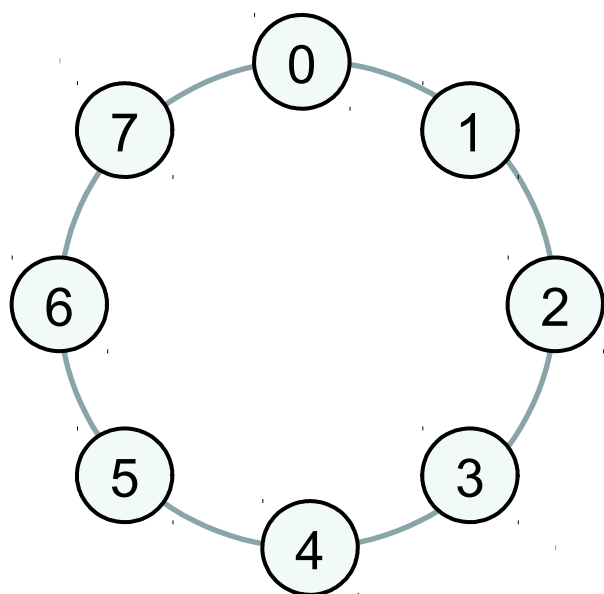
- logical process arrangement in topological patterns such as 2D or 3D grid; more generally, the logical process arrangement is described by a graph.

Virtual process topology .vs. topology of the underlying, physical hardware:

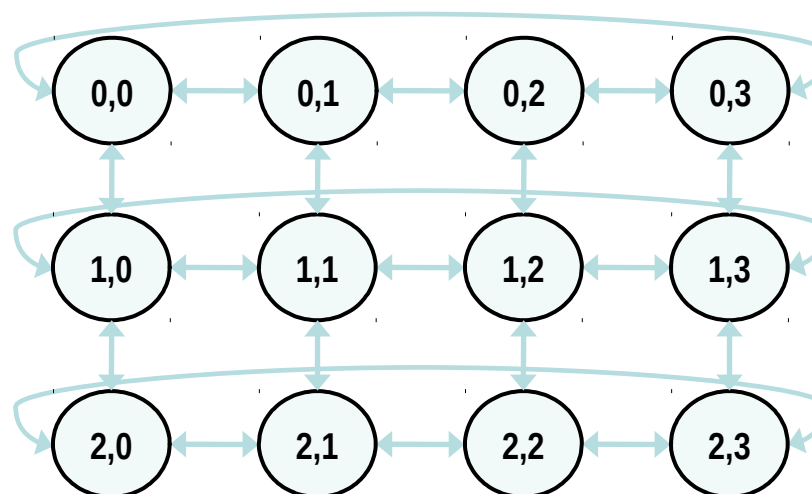
- virtual topology can be exploited by the system in the assignment of processes to physical processors, if this helps to improve the communication performance on a given machine.
- the description of the virtual topology depends only on the application, and is machine-independent.



Virtual Topology - Examples



RING



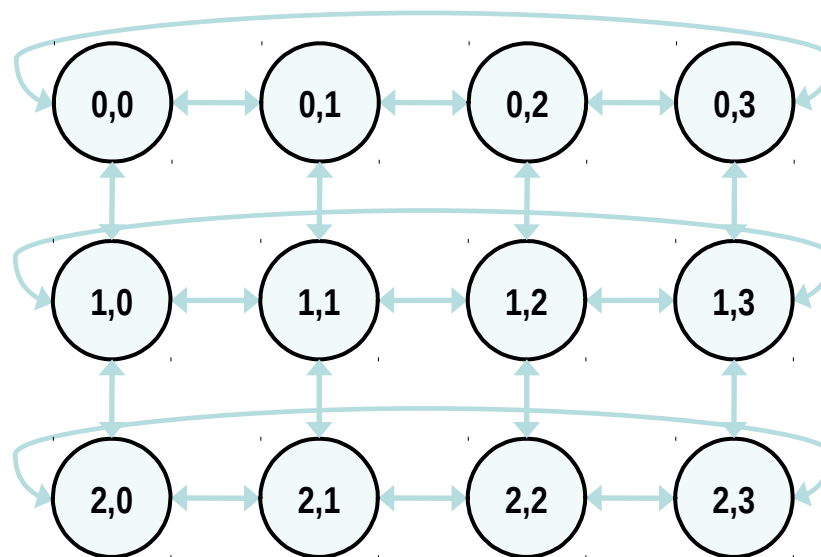
**2D-GRID WITH
PERIODIC
BOUNDARY
CONDITIONS₅**



Cartesian Topology

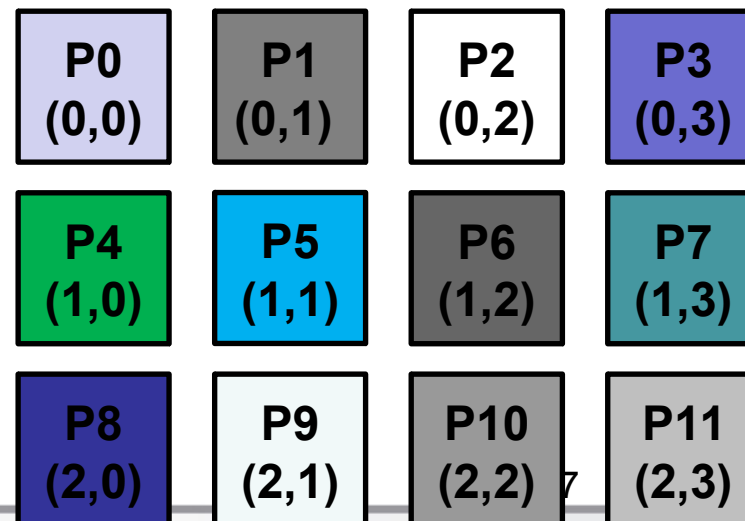
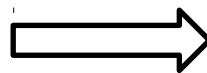
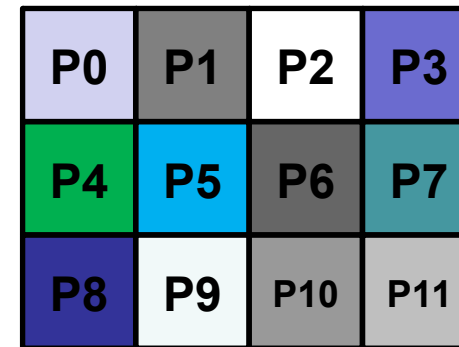
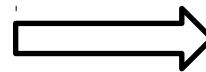
A grid of processes is easily described with a cartesian topology:

- each process can be identified by cartesian coordinates
- periodicity can be selected for each direction
- communications are performed along grid dimensions only





Example: 2D Domain decomposition





Cartesian Constructor

```
MPI_CART_CREATE(comm_old, ndims, dims, periods, reorder,  
comm_cart)
```

```
IN comm_old:    input communicator (handle)  
IN ndims:  number of dimensions of Cartesian grid (integer)  
IN dims:  integer array of size ndims specifying the number of  
processes in each dimension  
IN periods: logical array of size ndims specifying whether the  
grid is periodic (true) or not (false) in each dimension  
IN reorder: ranking may be reordered (true) or not (false)  
OUT comm_cart: communicator with new Cartesian topology (handle)
```

- Returns a handle to a new communicator to which the Cartesian topology information is attached.
- Reorder:
 - false: the rank of each process in the new group is identical to its rank in the old group.
 - True: the processes may be reordered, possibly so as to choose a good embedding of the virtual topology onto physical machine.
- If cart has less processes than starting communicator, left over processes have MPI_COMM_NULL as return



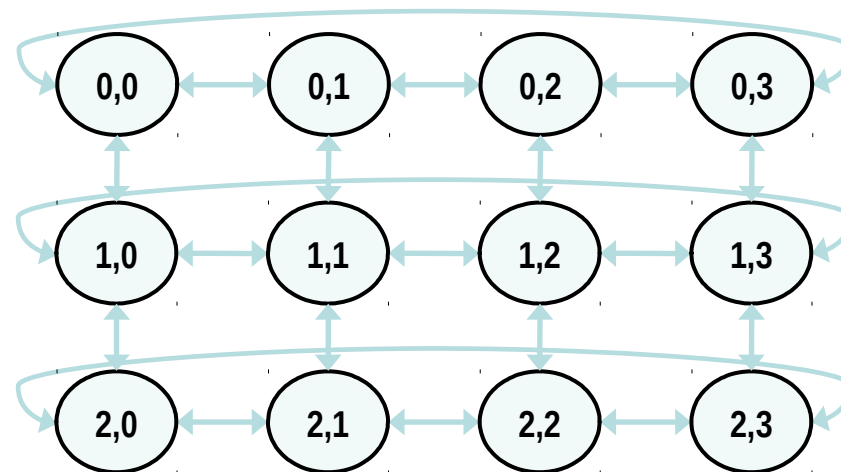
How to create a Cartesian Topology

```
#include <mpi.h>

int main(int argc, char *argv[])
{
    MPI_Comm cart_comm;
    int dim[] = {4, 3};
    int period[] = {1, 0};
    int reorder = 0;

    MPI_Init(&argc, &argv);

    MPI_Cart_create(MPI_COMM_WORLD, 2, dim, period, reorder,
    &cart_comm);
    ...
}
```





Cartesian Topology Utilities

MPI_Dims_Create:

- compute optimal balanced distribution of processes per coordinate direction with respect to:
 - a given dimensionality
 - the number of processes in a group
 - optional constraints

MPI_Cart_coords:

- given a rank, returns process's coordinates

MPI_Cart_rank:

- given process's coordinates, returns the rank

MPI_Cart_shift:

- get source and destination rank ids in SendRecv operations



Rank -> Coordinate: MPI_Cart_coords

MPI_CART_COORDS(comm, rank, maxdim, coords)

IN comm: communicator with Cartesian structure

IN rank: rank of a process within group of comm

IN maxdims: length of vector coords in the calling program

OUT coords: integer array (of size ndims) containing the Cartesian coordinates of specified process

For each MPI process in Cartesian communicator, the coordinate within the cartesian topology are returned



Usage of MPI_Cart_coords

```
. . .
ndim = (int*)calloc(dim,sizeof(int));
ndim[0] = row; ndim[1] = col;

period = (int*)calloc(dim,sizeof(int));
period[0] = period[1] = 0;

reorder = 0;

// 2D grid creation
MPI_Cart_Create(MPI_COMM_WORLD,dim,ndim,period,reorder, &comm_grid);
MPI_Comm_rank(comm_grid,&menum_grid);

// Coordinate of each mpi rank within the cartesian communicator
MPI_Cart_coords(comm_grid,menum,dim,coordinate);

printf("Procs %d coordinates in 2D grid (%d,%d)
\n",menum,*coordinate,* (coordinate+1));
. . .
}
```



Sendrecv with Cartesian Topologies: `MPI_Cart_shift`

`MPI_CART_SHIFT(comm, direction, disp, rank_source, rank_dest)`

IN `comm`: communicator with Cartesian structure

IN `direction`: coordinate dimension of shift

IN `disp`: displacement (>0: upwards shift; <0: downwards shift)

OUT `rank_source`: rank of source process

OUT `rank_dest`: rank of destination process

Returns the shifted source and destination ranks, given a shift direction and amount



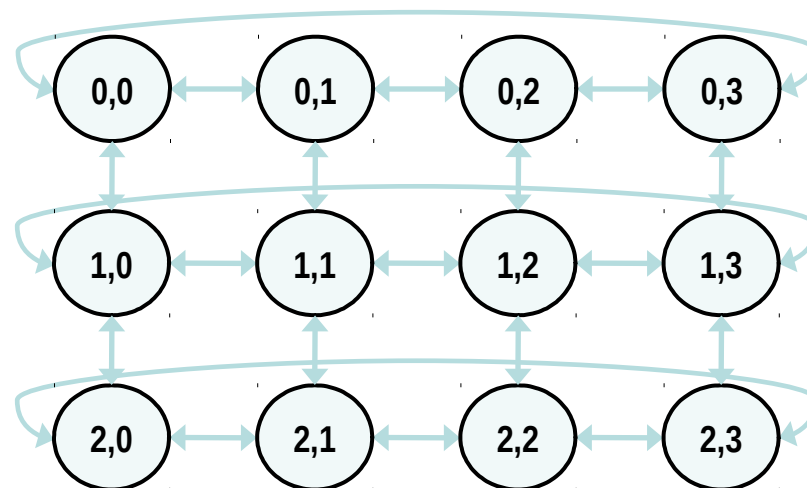
Sendrecv with 2D Cartesian Topologies

...

```
int dim[] = {4, 3};  
int period[] = {1, 0};  
MPI_Comm grid_comm;
```

```
MPI_Cart_create(MPI_COMM_WORLD, 2,  
               dim, period, 0, &grid_comm);
```

```
int source, dest;  
for (int dimension = 0; dimension < 2; dimension++) {  
    for (int versus = -1; versus < 2; versus+=2;) {  
        MPI_Cart_shift(ring_comm, dimension, versus, &source, &dest);  
        MPI_Sendrecv(buffer, n, MPI_INT, source, stag,  
                     buffer, n, MPI_INT, dest, dtag,  
                     grid_comm, &status);  
    }  
}
```



Destructors

The communicators and groups from a process' viewpoint are just handles.

Like all handles, there is a limited number available: you could (in principle) run out!

MPI_GROUP_FREE(group, ierr)

MPI_COMM_FREE(comm, ierr)